NANYANG TECHNOLOGICAL UNIVERSITY

IE2104 – Digital Electronics

School of Electrical & Electronic Engineering

Lecturer: Prof. Chang Chip Hong

Office: S2-B2c-97

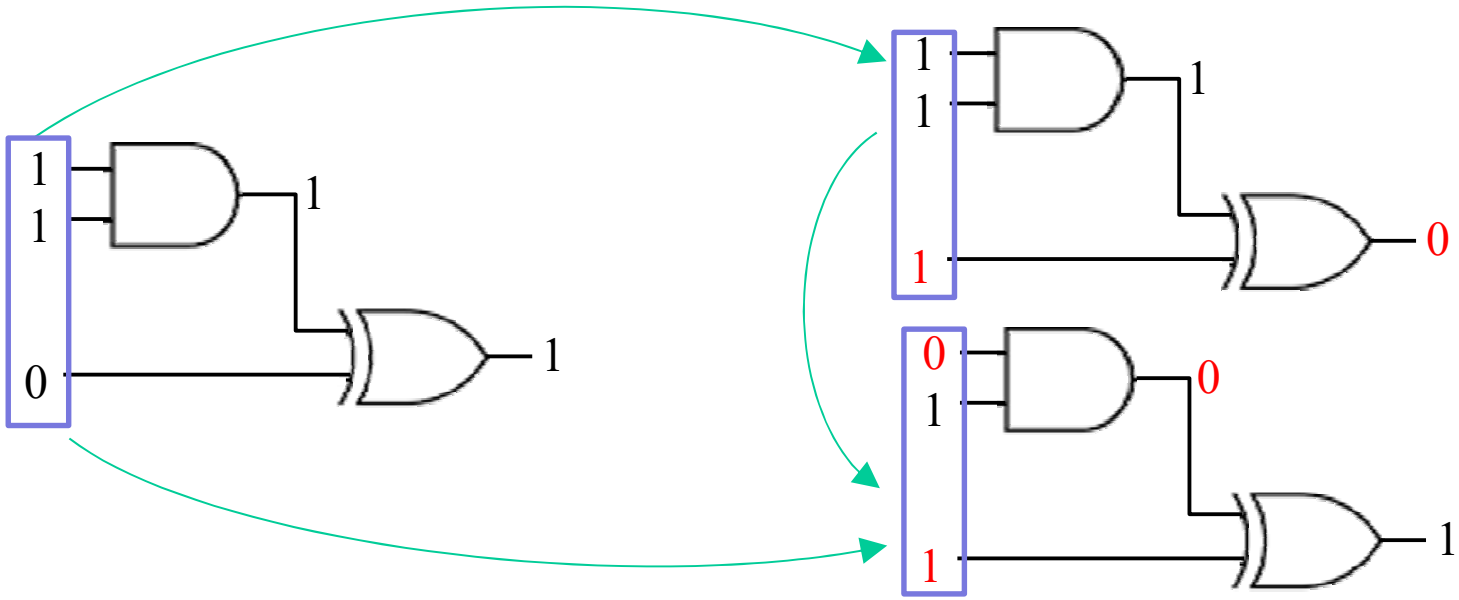
Email: echchang@ntu.edu.sg

Topic : Sequential Circuits

- Latches & flip-flops
- Synchronous state machine analysis
- Synchronous state machine design
- Timing parameters
- Counters
- Shift registers
- Memory

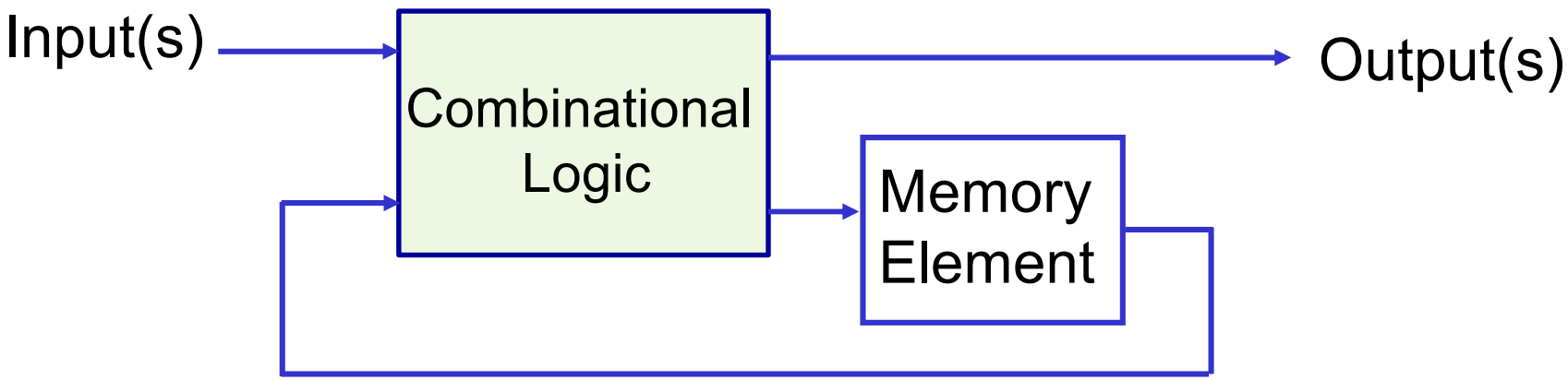
Combinational Circuits

- Each output depends only on present values of inputs and not on past values. Therefore, the output is strictly a function of the inputs.
- Examples: Adders, subtractors, encoders, decoders, multiplexers, demultiplexers, comparators, PLA, PAL, etc.
- Formed by connecting logic gates (like AND, OR, NOR, XOR, NAND, NOR, XNOR) without any feedback loop.



Sequential Circuits

- Outputs depend on both present and past input values. Past input values determine present state of the circuit.
- To remember the previous state, the sequential circuit must have some form of 'memory'.



Block Diagram of Sequential Circuit

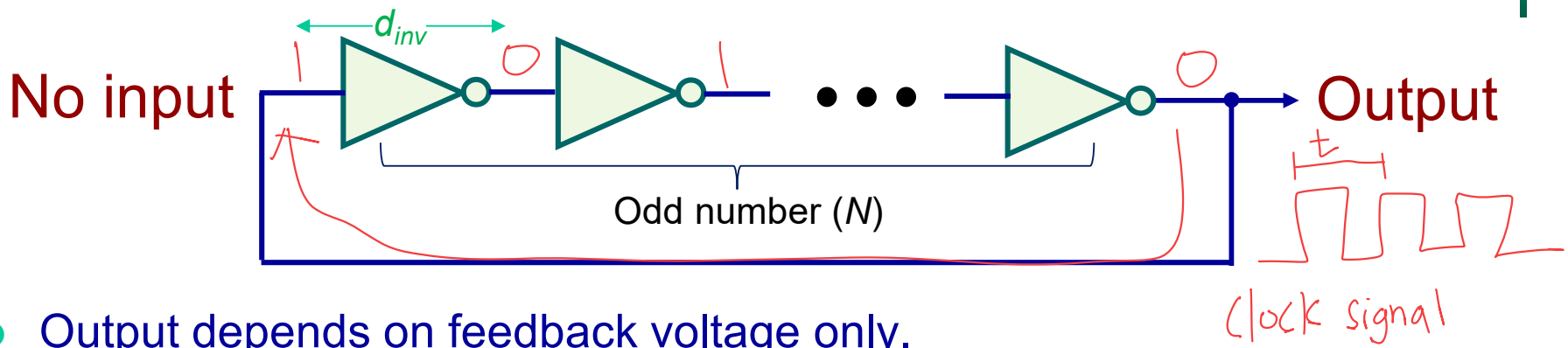
Storage Elements

- A storage element can maintain a binary state indefinitely (as long as the power is delivered to the circuit), until it is directed by an input signal to switch states.
- **Latch:** a memory element whose excitation input signals control the state of the device. A latch checks input continuously and changes the output whenever there is a change in input.
- **Flip-flop:** differs from a latch in that it has a control signal called a **clock**. Any change to its output can only be initiated by the clock signal.





Inverter Ring (Odd No. of Inverters)



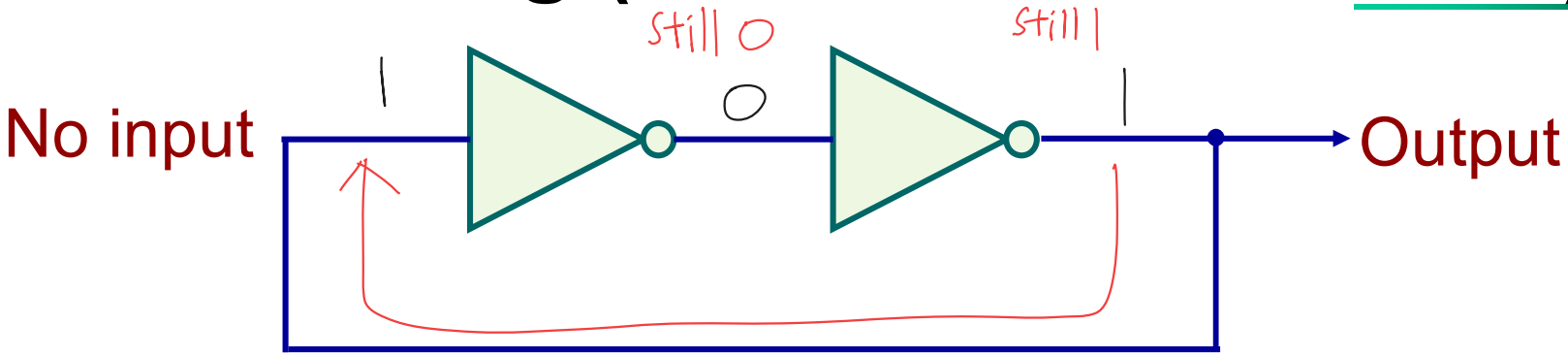
- Output depends on feedback voltage only.
- When the output of an inverter chain with an odd number of inverters is fed back to the input, it appears inverted on the output after a short time (propagation delay).
- The new state of voltage which feeds back to the input causes a similar inversion.
- The output voltage oscillates between logic '0' and '1'.

$$f_{osc} = \frac{1}{2 \times N \times d_{inv}}$$

where N is the number of inverters, and d_{inv} is the delay of an inverter.

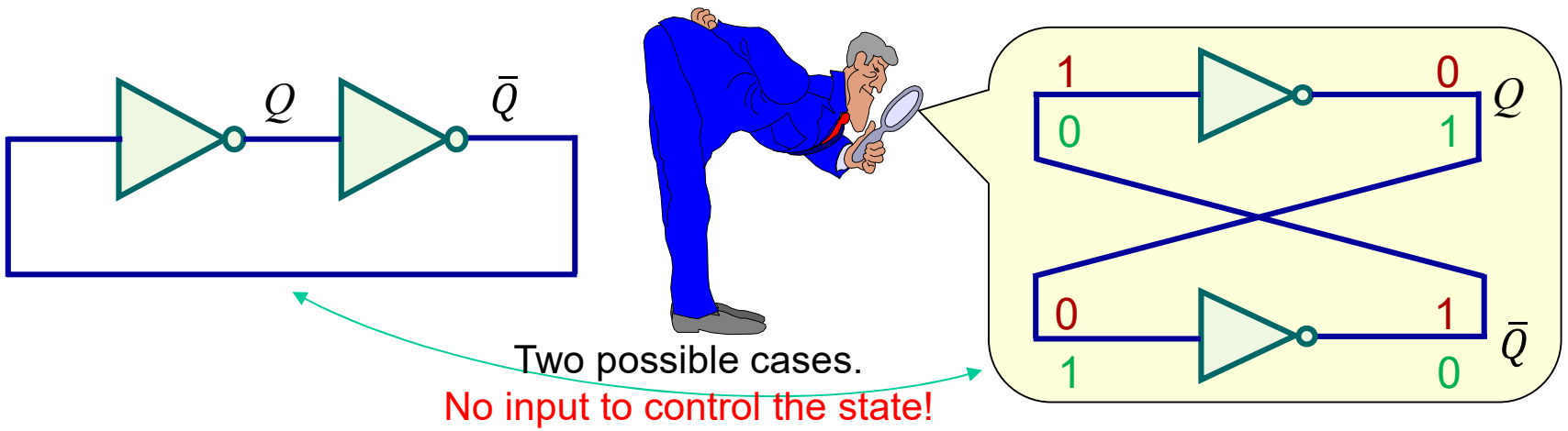
- A single inverter ring will not oscillate. The output is half-way between supply voltage and ground. (Barkhausen stability criterion)

Inverter Ring (Even No. of Inverters)

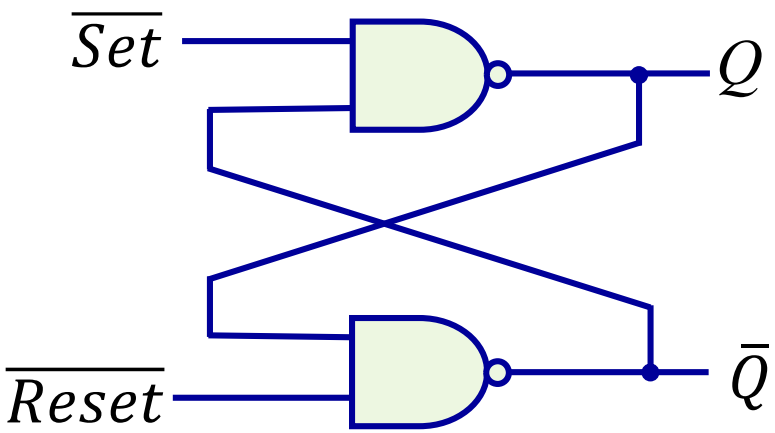


- When powered on, the output of each inverter will go through a period of instability.
- One output settles to a high level and the other to a low level ⇨ A bistable circuit.
- The stable output state is unpredictable.
- Circuit exhibits a primitive form of "memory", it remembers the resolution of the initial voltage applied on the inputs of the inverters.

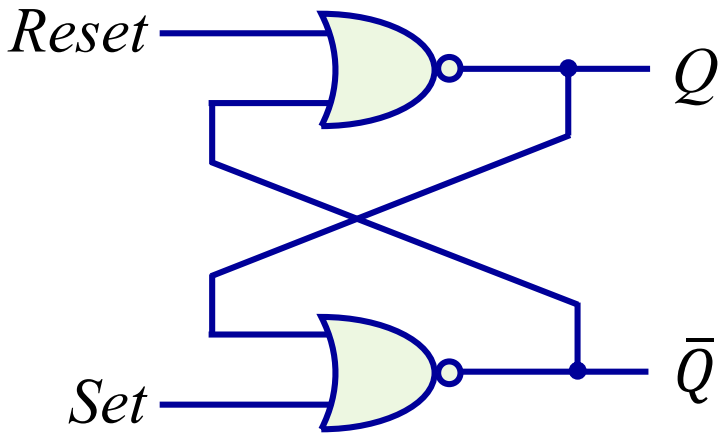
Cross-coupled Cells



Feedback circuits with external inputs allow designers to control the value stored in the memory ⇒ Basic Binary cells



Cross-coupled NAND



Cross-coupled NOR

Basic Binary Cell

- Binary cell is the basic building block of memory, it is used to store a single bit of information.

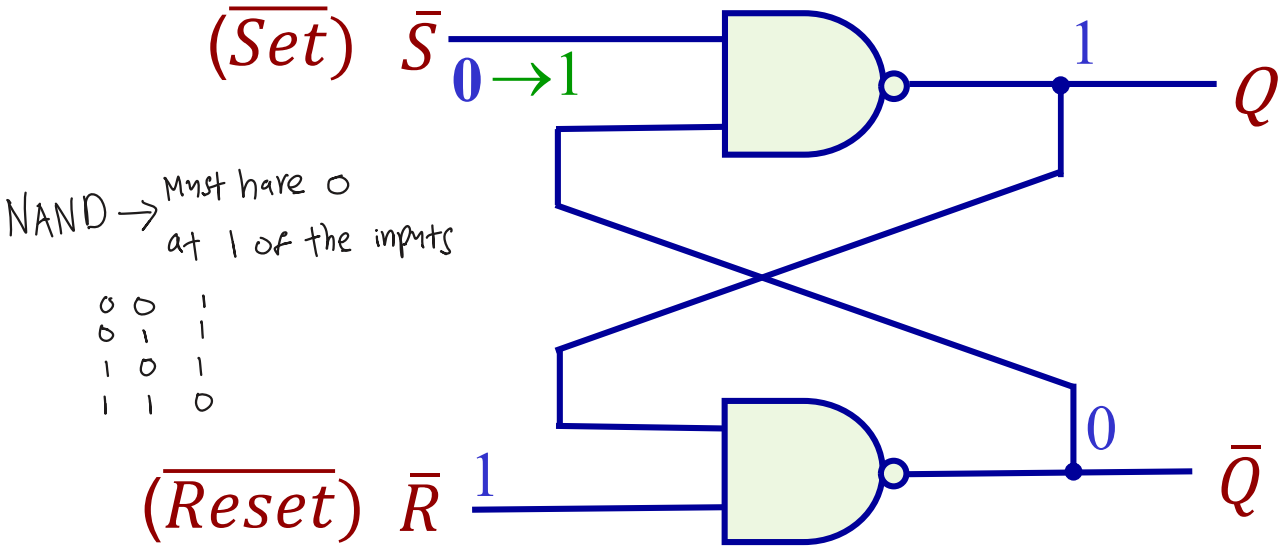
SET condition ⇒ stores a ‘1’

RESET condition ⇒ stores a ‘0’

REST condition ⇒ state unchanged

- Two types of basic binary cells:
  - Cross-coupled NAND
  - Cross-coupled NOR

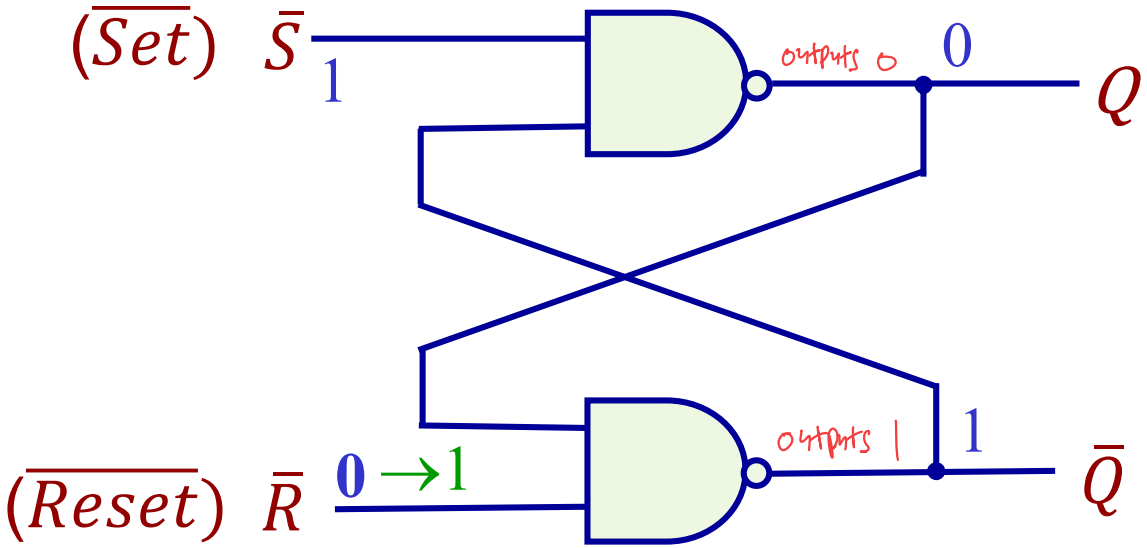
Cross-coupled NAND Cell



(a) Set state (set condition) Active low

- When input $\bar{S}$ is asserted (**0**) and input $\bar{R}$ is not asserted (1), then Q = 1 and $\bar{Q}$ = 0.
  ∴ binary cell is loaded with a binary bit "1" (i.e., SET).
- If $\bar{S}$ is de-asserted (1) thereafter, the cell condition does not changed (i.e., REST or HOLD).

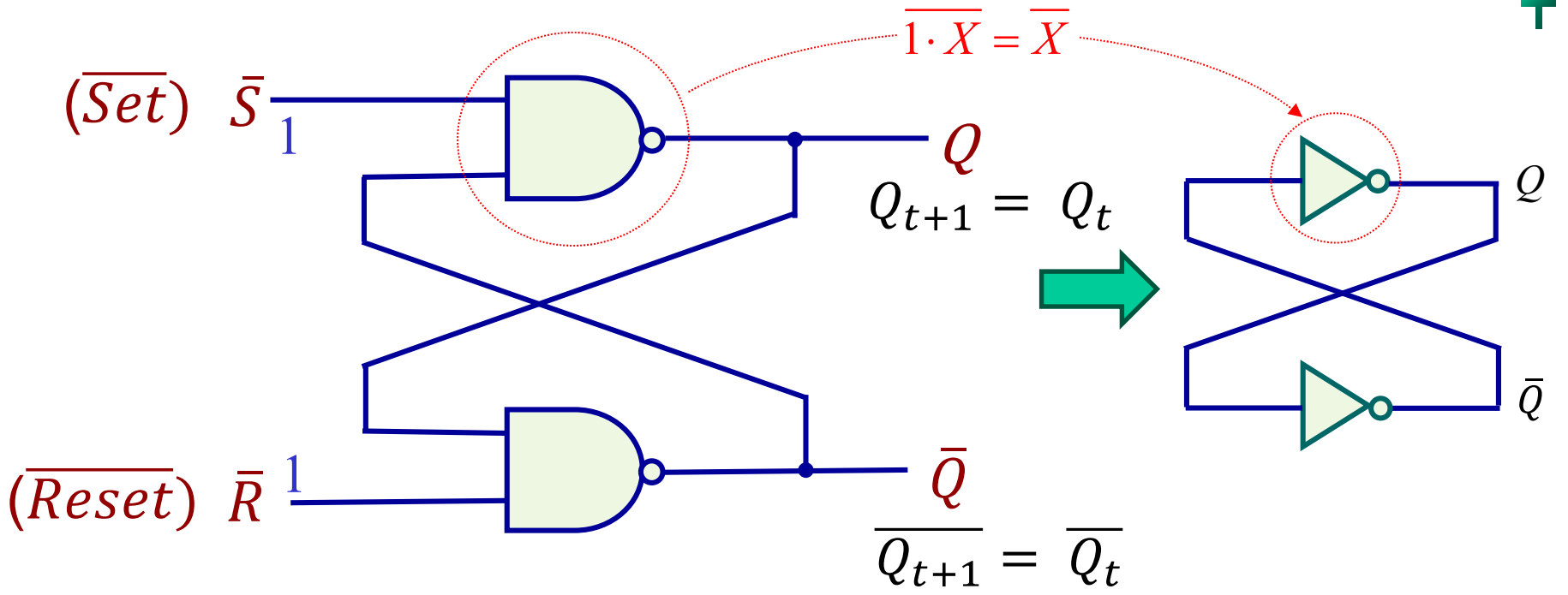
Cross-coupled NAND Cell



(b) Reset state (Reset condition)

- When input $\bar{S}$ is not asserted (1) and input $\bar{R}$ is asserted (**0**), then $\bar{Q}$ = 1 and Q = 0.
  ∴ binary cell is loaded with a binary bit "0" (i.e., RESET).
- If $\bar{R}$ is de-asserted thereafter, the cell condition does not changed (i.e., REST or HOLD).

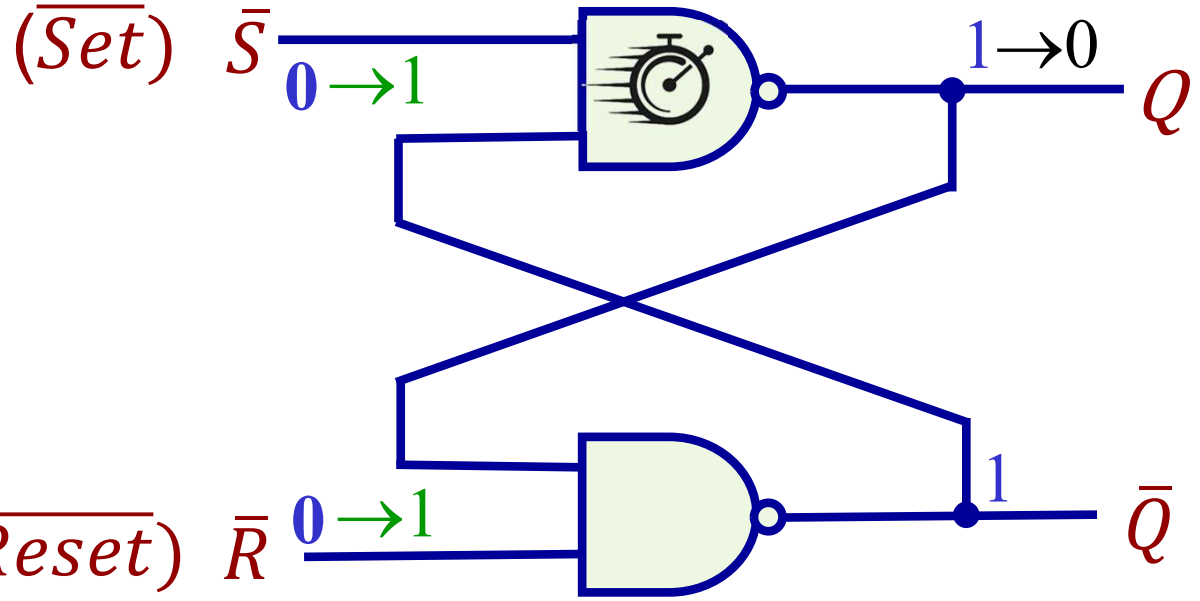
Cross-coupled NAND Cell



(c) Rest state (Rest condition)

- When inputs $\bar{S}$ and $\bar{R}$ are both not asserted (1), then Q and $\bar{Q}$ are both unchanged.

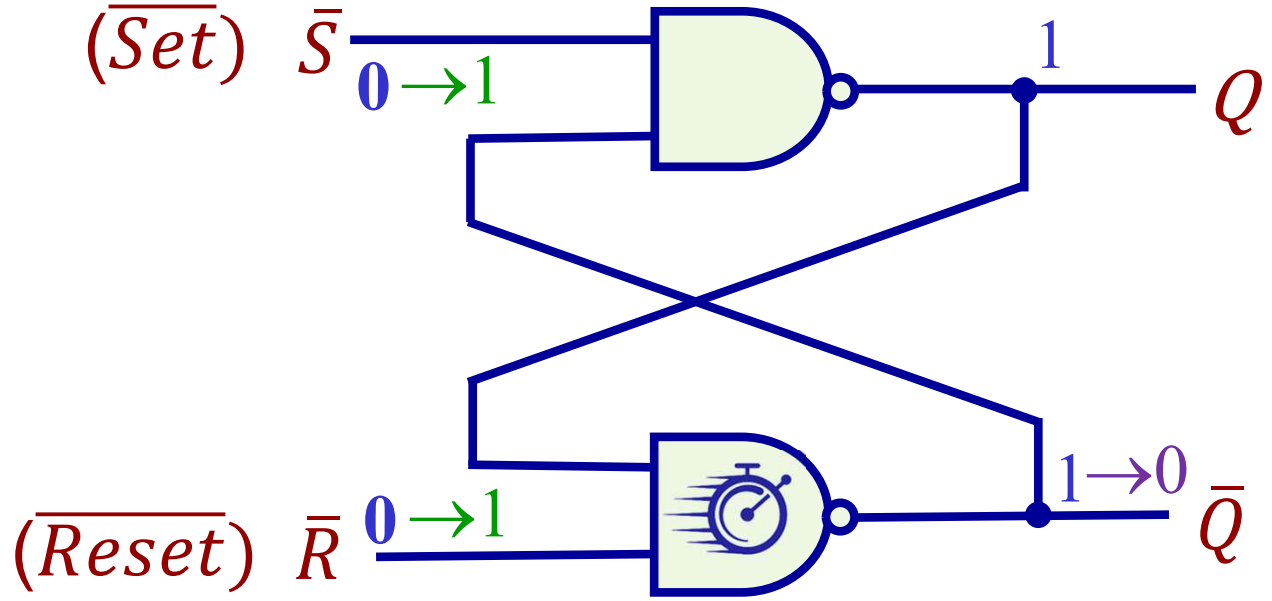
Cross-coupled NAND Cell



(d) Forbidden State (Not Allowed condition)

- When inputs $\bar{S}$ and $\bar{R}$ are asserted (0) simultaneously, then Q and $\bar{Q}$ are both high $\Rightarrow$ Contradictory logic.
- When both inputs are released (1) simultaneously, the outputs are indeterministic due to the race between Q and $\bar{Q}$.
  - Q = 0 if upper gate is faster.

Cross-coupled NAND Cell

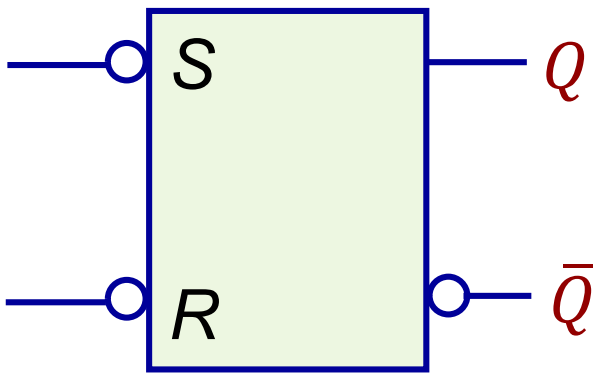


(d) Forbidden State (Not Allowed condition)

- When inputs $\bar{S}$ and $\bar{R}$ are asserted (0) simultaneously, then Q and $\bar{Q}$ are both high $\Rightarrow$ Contradictory logic.
- When both inputs are released (1) simultaneously, the outputs are indeterministic due to the race between Q and $\bar{Q}$.
  - Q = 0 if upper gate is faster.
  - $\bar{Q}$ = 0 if lower gate is faster.

Cross-coupled NAND Cell

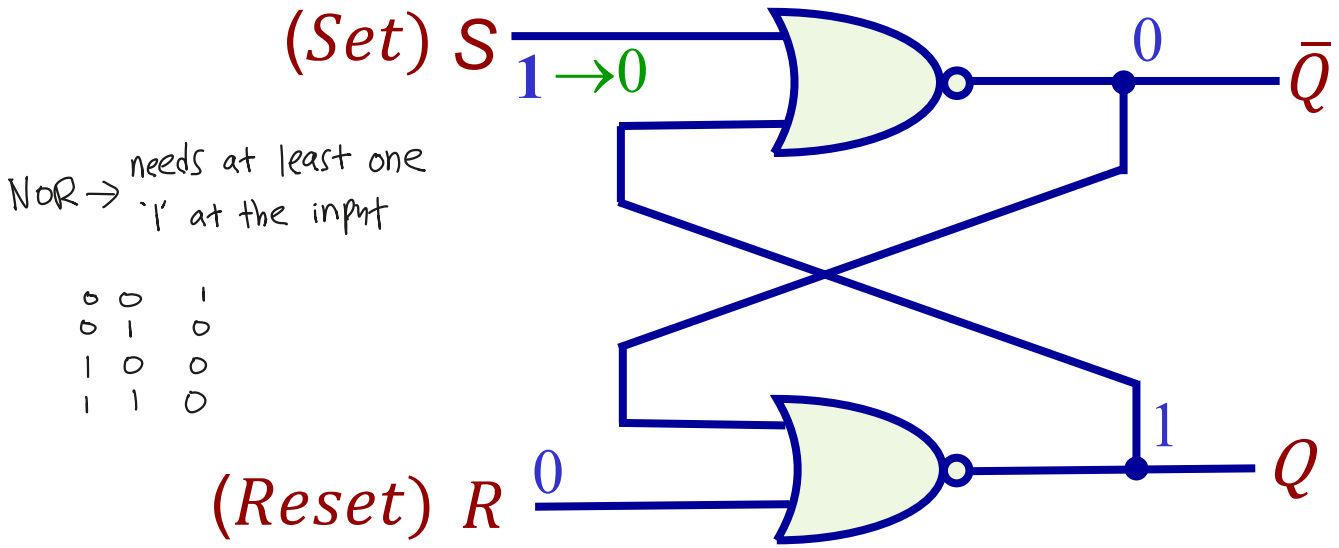
Symbol



Function Table

$\bar{S}$	$\bar{R}$	Q	$\bar{Q}$	
0	1	1	0	Set
1	1	1	0	Rest
1	0	0	1	Reset
1	1	0	1	Rest
0	0	1	1	Not allowed

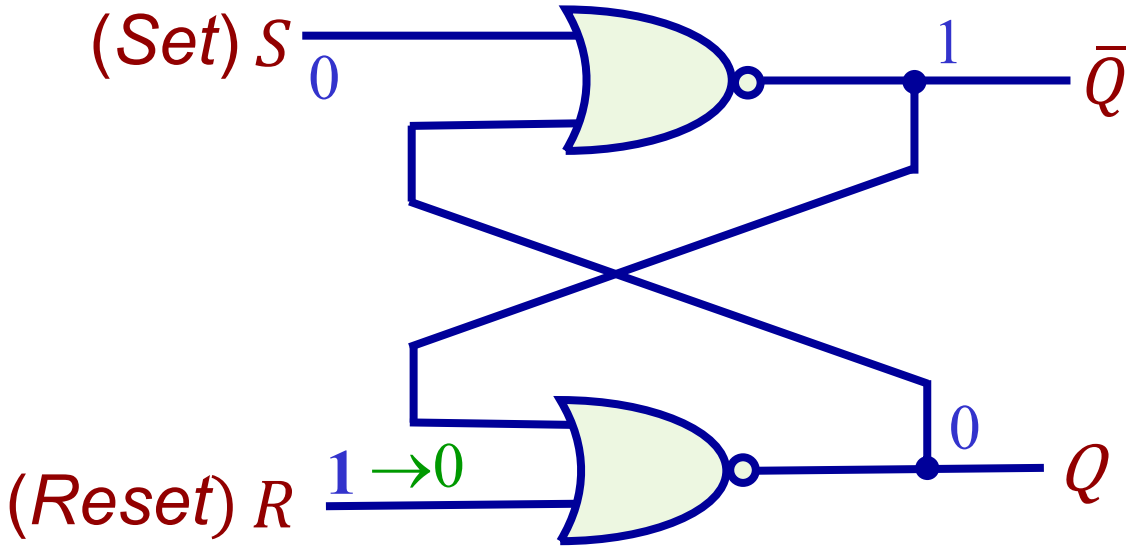
Cross-coupled NOR Cell



(a) Set state (set condition)

- When input S is asserted (**1**) and input R is not asserted (0), then $\bar{Q}$ = 0 and Q = 1.
  ∴ binary cell is loaded with a binary bit "1" (i.e., SET).
- If S is de-asserted (0) thereafter, the cell condition does not changed (i.e., REST or HOLD).

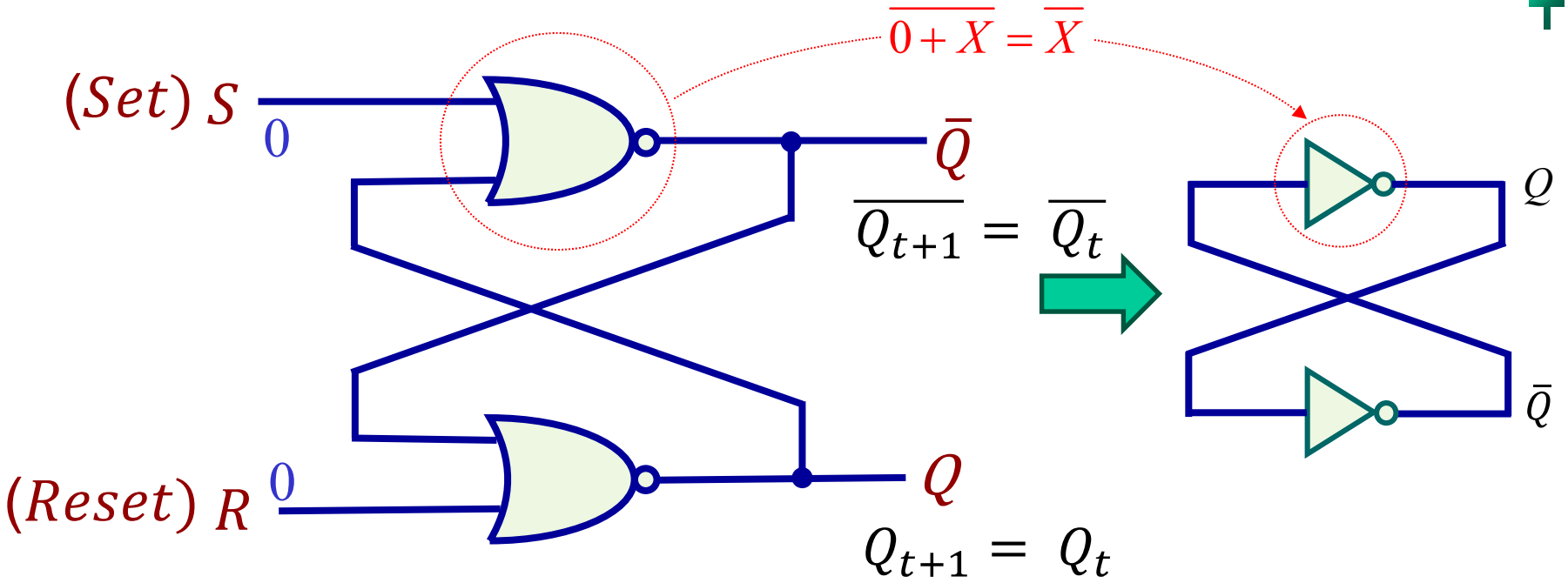
Cross-coupled NOR Cell



(b) Reset state (Reset condition)

- When input S is not asserted (0) and input R is asserted (**1**), then Q = 0 and $\bar{Q}$ = 1.
  ∴ binary cell is loaded with a binary bit "0" (i.e., RESET).
- If R is de-asserted thereafter, the cell condition does not changed (i.e., REST or HOLD).

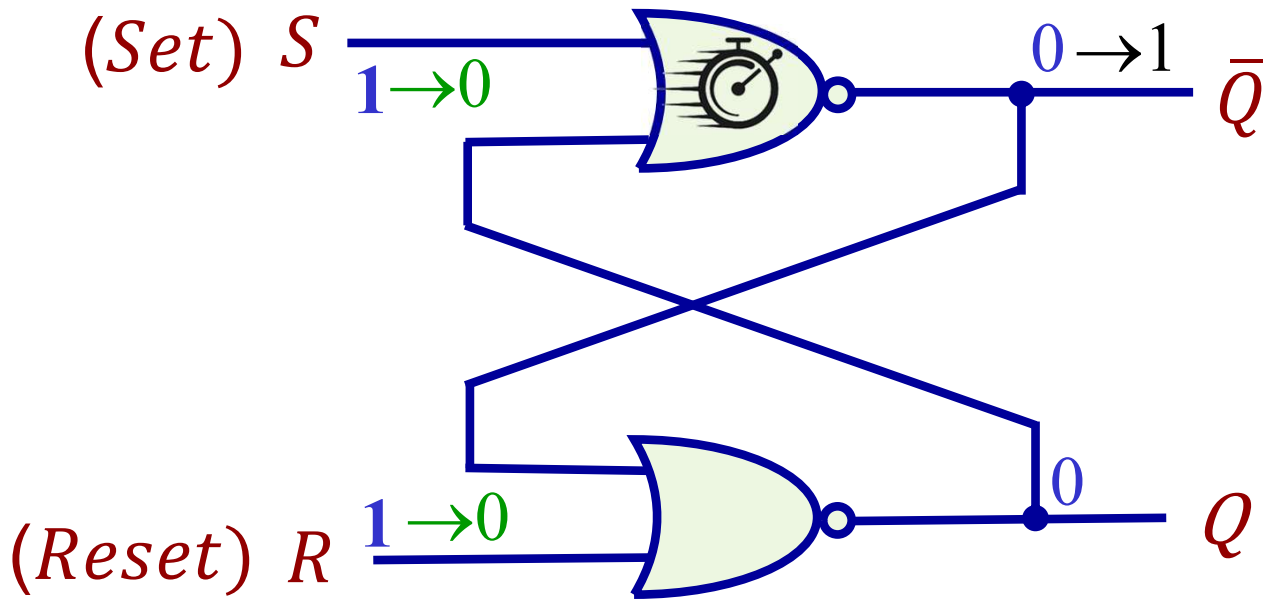
Cross-coupled NOR Cell



(c) Rest state (Rest condition)

- When inputs S and R are both not asserted (0), then Q and $\bar{Q}$ are both unchanged.

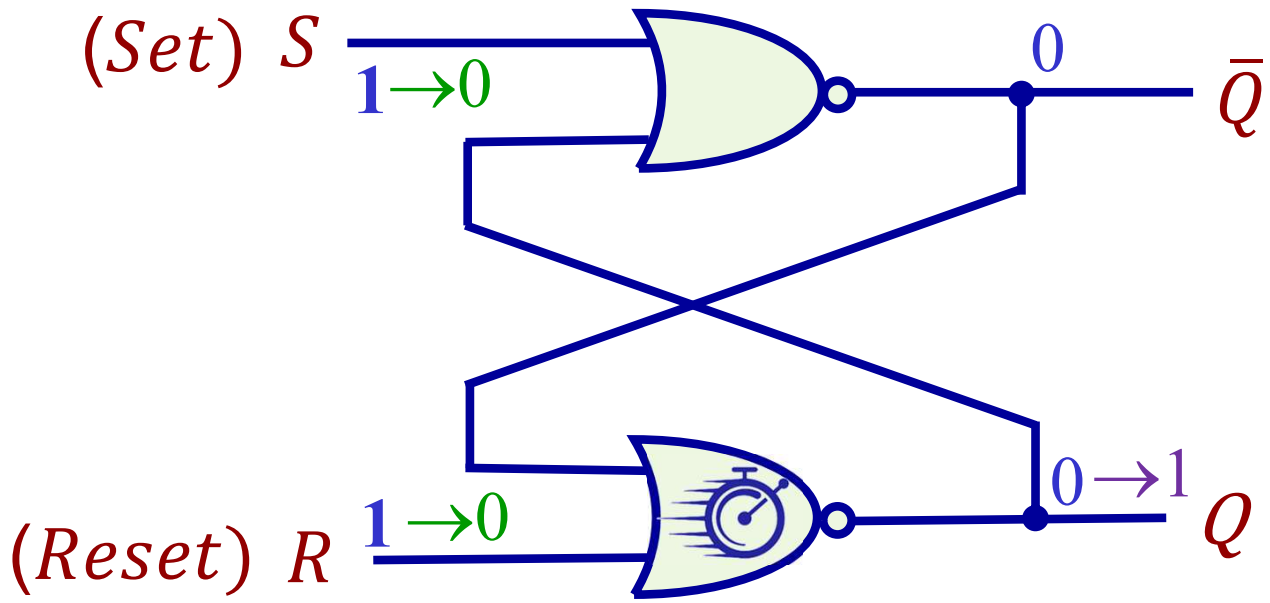
Cross-coupled NOR Cell



(d) Forbidden State (Not Allowed condition)

- When inputs S and R are asserted (1) simultaneously, then Q and $\bar{Q}$ are both low $\Rightarrow$ Contradictory logic.
- When both inputs are released (0) simultaneously, the outputs are indeterministic due to the race between Q and $\bar{Q}$.
  - $\bar{Q}$ = 1 if upper gate is faster.

Cross-coupled NOR Cell

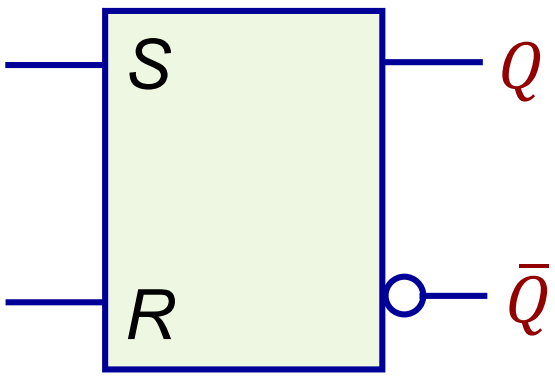


(d) Forbidden State (Not Allowed condition)

- When inputs S and R are asserted (1) simultaneously, then Q and $\bar{Q}$ are both low ⇒ Contradictory logic.
- When both inputs are released (0) simultaneously, the outputs are indeterministic due to the race between Q and $\bar{Q}$.
  - $\bar{Q}$ = 1 if upper gate is faster.
  - Q = 1 if lower gate is faster.

Cross-coupled NOR Cell

Symbol

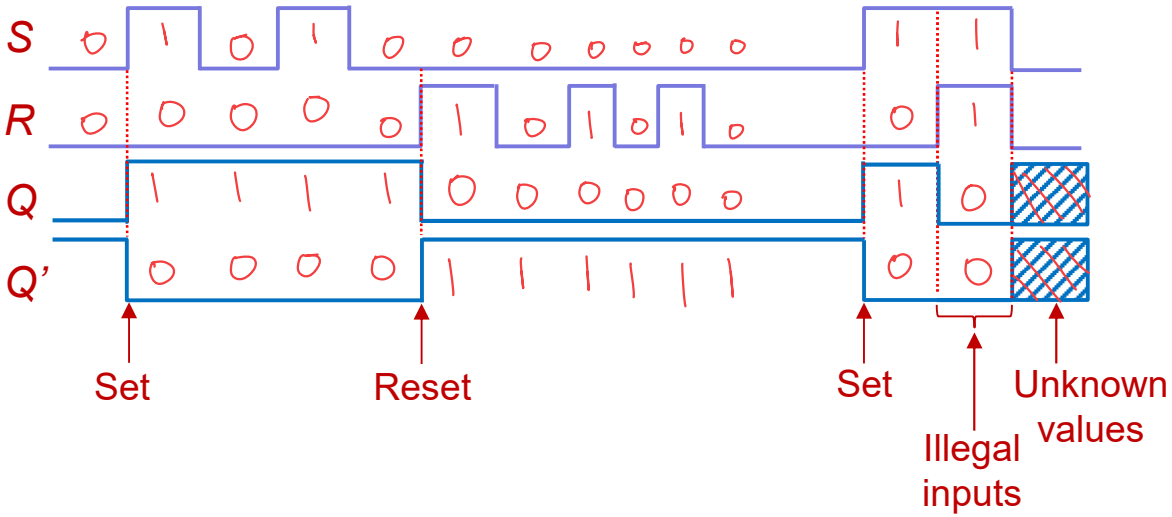


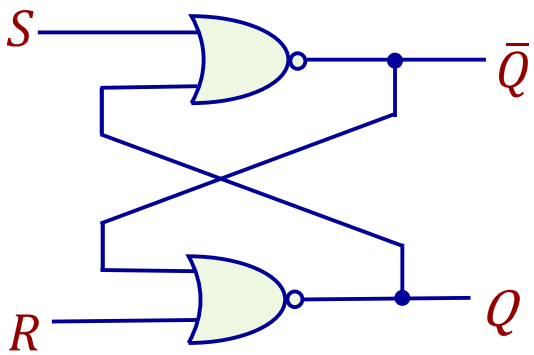
Function Table

S	R	Q	$\bar{Q}$	
1	0	1	0	Set
0	0	1	0	Rest
0	1	0	1	Reset
0	0	0	1	Rest
1	1	0	0	Not allowed

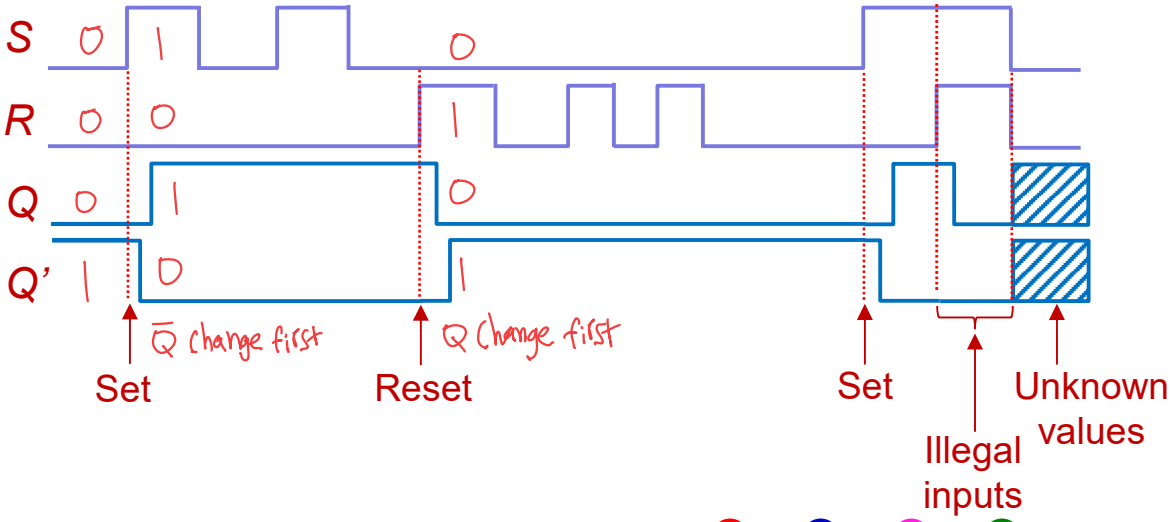
Timing Diagram of SR Latch

Ideal timing with zero-gate-delay

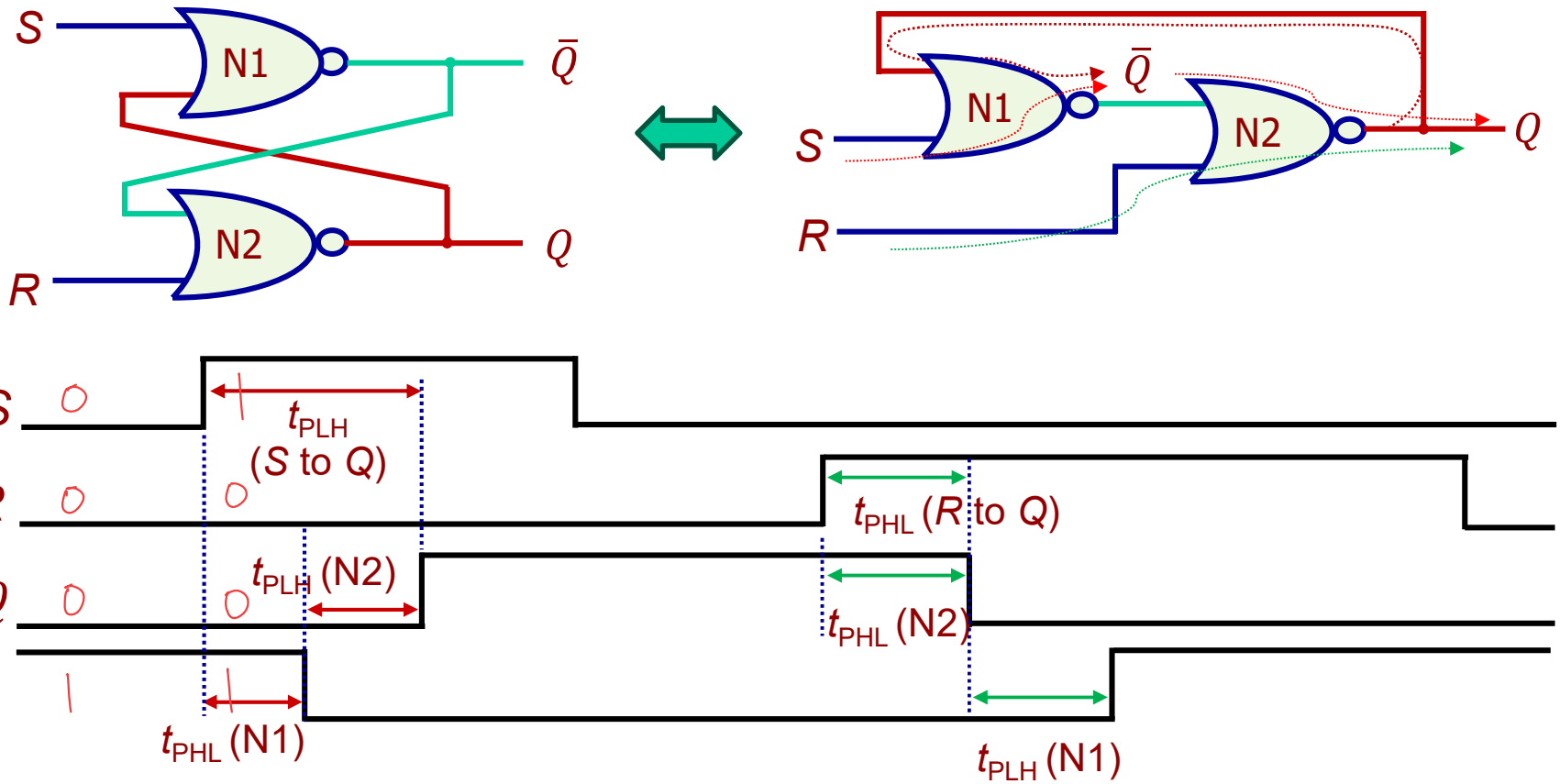




Actual timing with non-zero gate delay



Propagation Delay of SR Latch



t_{PHL}: time between an input change and a corresponding *high-to-low output* transition.
t_{PLH}: time between an input change and a corresponding *low-to-high output* transition.
t_{PLH} and **t_{PHL}** represent the sum of the propagation delays through the gates between a given latch input and output, with separate delay parameters usually specified for each input/output pair.

Timing Behavior of SR Latch

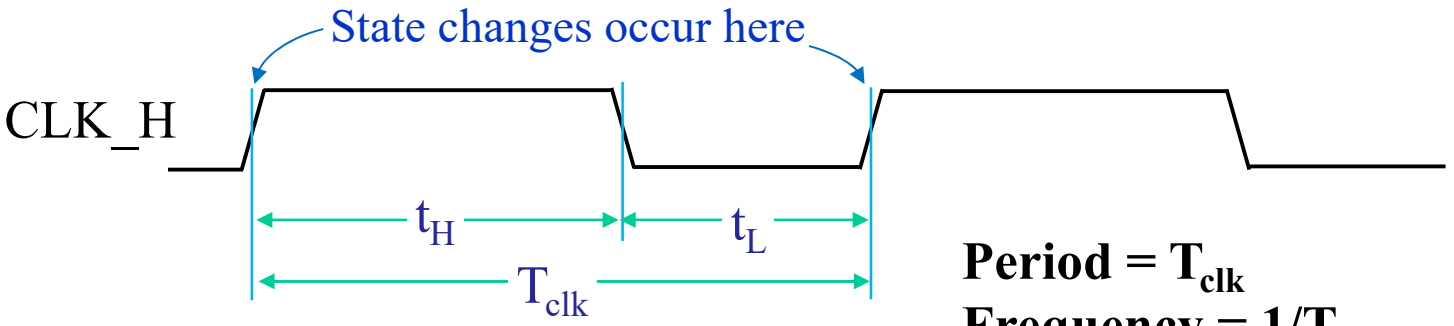
- When S changes from 0→1, $\bar{Q}$ changes from 1→0 after the propagation delay t_{PHL} through NOR gate N1.
- Then, Q changes from 0→1 after t_{PLH} through gate N2.
- $\bar{Q}$ changes before Q when setting a cross-coupled NOR SR latch.
- Therefore, t_{PHL} from S to $\bar{Q}$ takes one gate delay, whereas t_{PLH} from S to Q takes two gate delays.
- A similar behavior exists between input R and the two outputs.
- When resetting, i.e., when R changes from 0→1, output Q changes before output $\bar{Q}$.
- Therefore, t_{PHL} from R to Q takes one gate delay, whereas t_{PLH} from R to $\bar{Q}$ takes two gate delays.

Clocked Latch or Flip-Flop

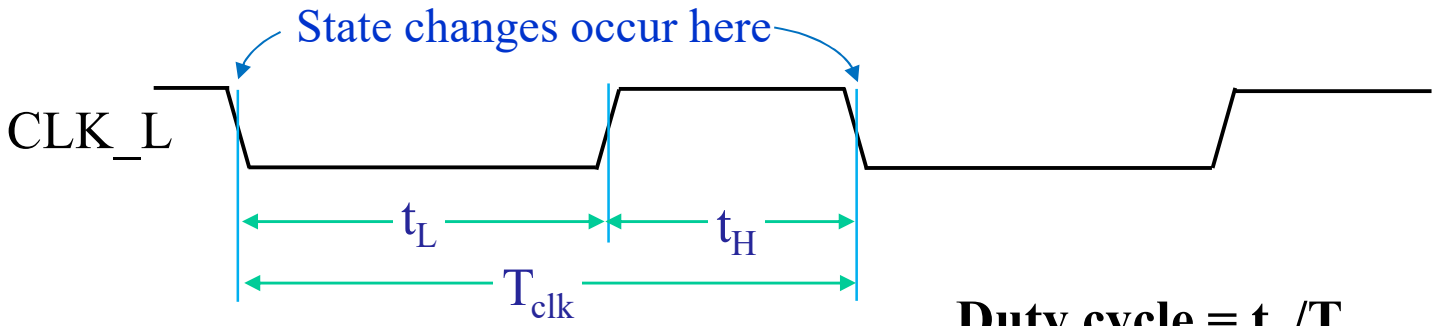
- A latch is a memory device that can store one of the two stable output states, has a pair of complementary outputs, and has one or more inputs that can cause the output state to change.
- The basic latch (cross-coupled NAND or NOR cell) as it stands is an ***asynchronous*** sequential circuit.
- It is a common practice to synchronize the operation of all latches by a common clock or pulse generator in ***synchronous*** sequential circuit.
- By adding some combinational circuits to the inputs of the basic latch, the latch can be made to respond to input levels during the occurrence of a ***clock*** pulse.

What Is a Clock Signal?

- Specifications: Period, Frequency, Duty cycle.

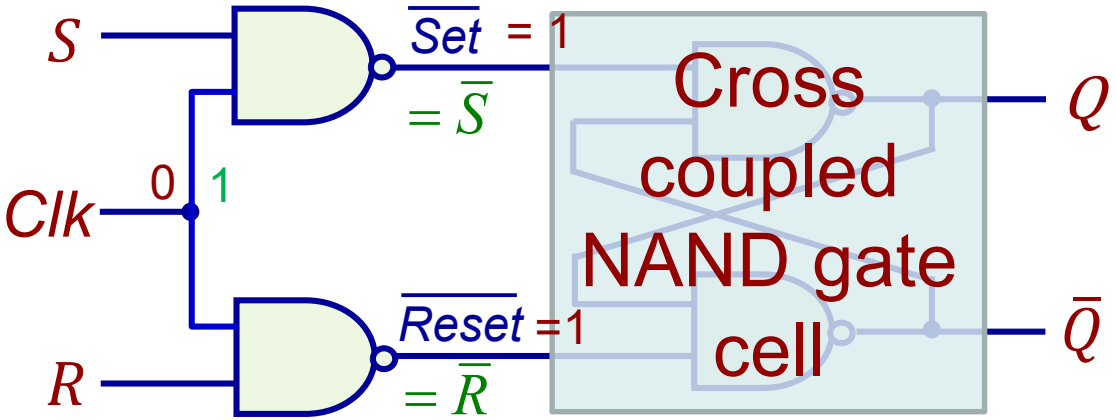


Period = T_{clk}
Frequency = $1/T_{clk}$
Duty cycle = t_H/T_{clk}



Duty cycle = t_L/T_{clk}

Clocked Set/Reset (SR Flip-Flop)



Clk	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	Q = 0, reset state
1	1	0	Q = 1, set state
1	1	1	indeterminate

Characteristic Table of SR FF

A Truth-Table for specifying the operational characteristic of the clocked flip-flop.

current value next value

S	R	Q_t	Q_{t+1}	Mode
0	0	0	0	Rest
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	X	Not Allowed
1	1	1	X	

Characteristic Equation of SR FF

The characteristic equation of the flip-flop specifies the value of the next state as a function of the present state and the inputs.

S	R	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

Q_t \ $S\,R$	00	01	11	10
0	0	0	X	1
1	1	0	X	1

$$Q_{t+1} = \bar{R}Q_t + S$$

For active high clock:

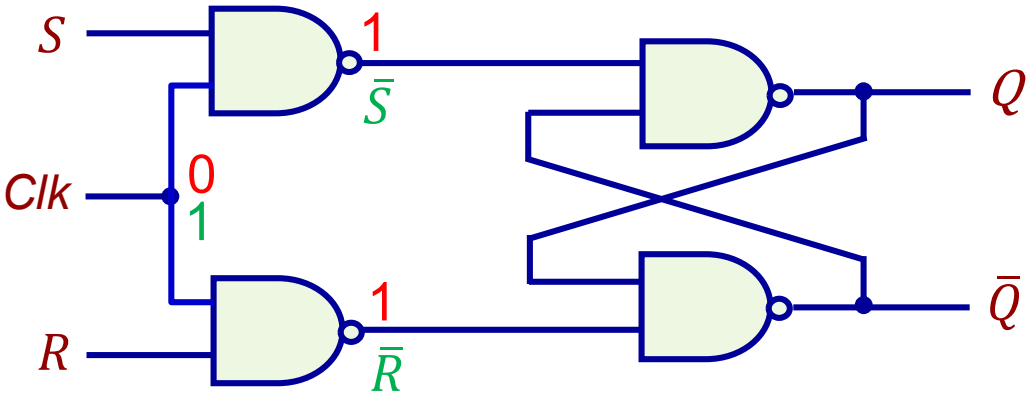
$$Q_{t+1} = \overline{Clk} \cdot Q_t + Clk \cdot \left(\bar{R}Q_t + S\right)$$

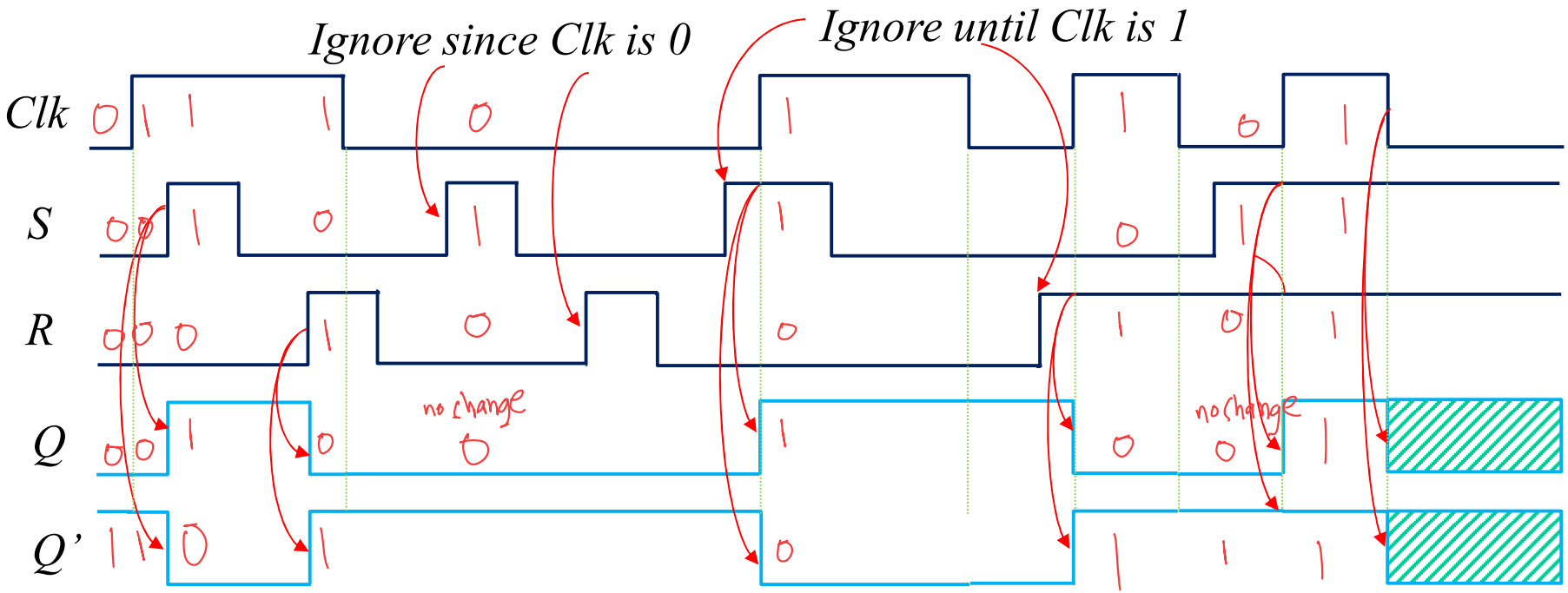
Excitation Table of SR FF

The excitation table is an important design aid derived from the characteristic table. It provides the desired inputs to produce a specific output transition.

Q_t	Q_{t+1}	S	R	
0	0	0	X	Rest 0 0 or Reset 0 1
0	1	1	0	
1	0	0	1	
1	1	X	0	Rest 0 0 or Set 1 0

Typical operation of SR Flop-Flop





Types of FFs

D Flip-Flop		
D	**Action**	**Next State**
0	Reset	0
1	Set	1

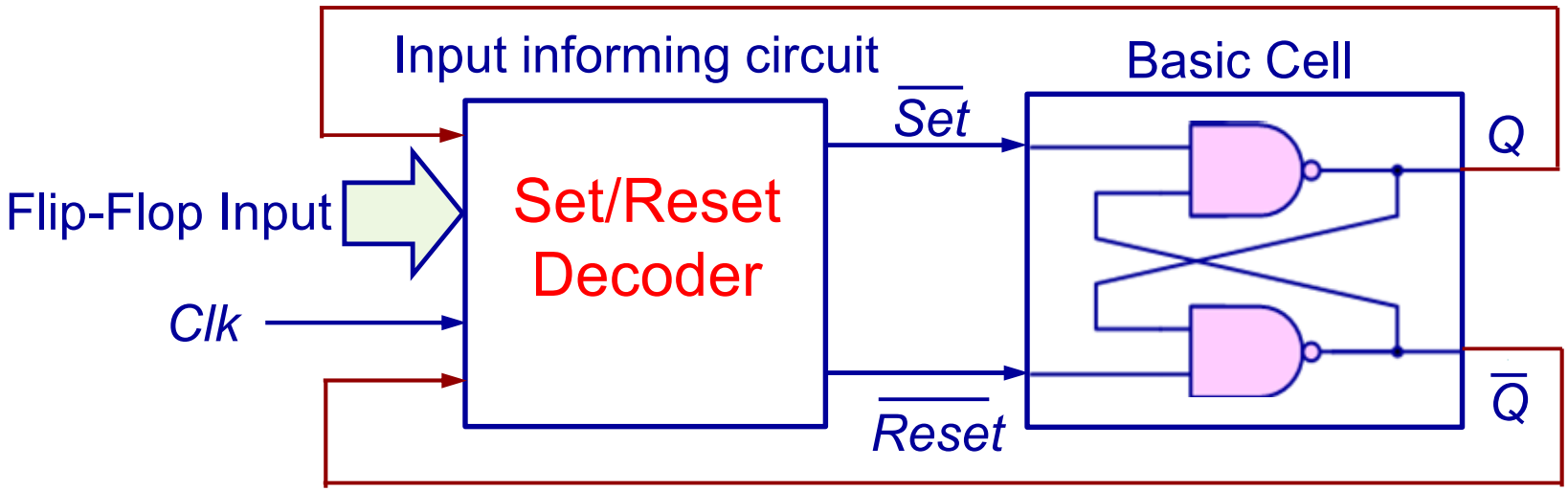
T Flip-Flop		
T	**Action**	**Next State**
0	Do nothing	Unchanged
1	Toggle	Change state (0→1, 1→0)

SR Flip-Flop			
Inputs		**Action**	**Next State**
S	**R**		
0	0	Do nothing	Unchanged
0	1	Reset	0
1	0	Set	1
1	1	Avoided	?

JK Flip-Flop			
Inputs		**Action**	**Next State**
J	**K**		
0	0	Do nothing	Unchanged
0	1	Reset	0
1	0	Set	1
1	1	Toggle	Change state (0→1, 1→0)

General Model of Flip-Flop

- Different flip-flops can be designed by adding input forming logic to basic cross-coupled NAND or NOR cell.
- The Input Forming Logic (IFL)
  - decodes the flip-flop inputs, including the clock pulse and the present state of the basic cell.
  - produces the required asserting output signals, SET or RESET to the basic cell, in synchrony with the clock, while not asserting SET and RESET at the same time.



D Flip-Flop from Clocked SR Latch

D Flip-Flop Characteristics

D	Q_t	Q_{t+1}	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

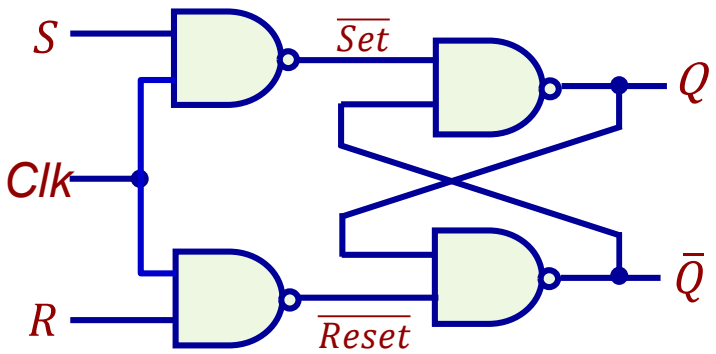
SR Flip-Flop excitation

Q_t \ D	0	1
0	0	1
1	0	X

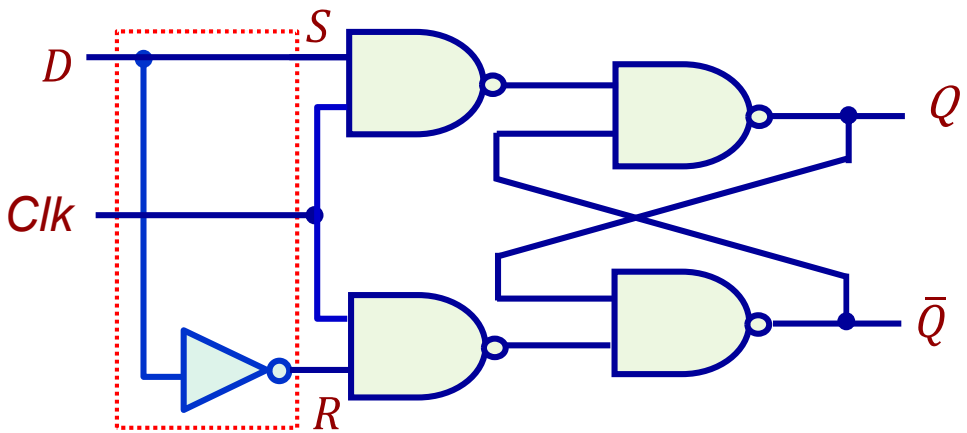
$S = D$

Q_t \ D	0	1
0	X	0
1	1	0

$R = \bar{D}$



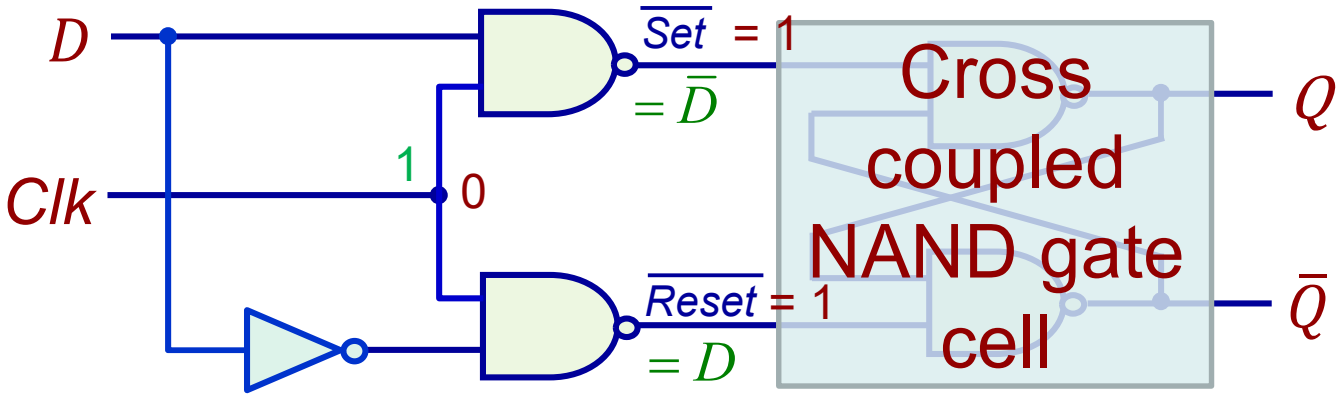
Clocked SR latch
a.k.a. SR flip-flop



IFL

D Flip-Flop (Clocked D latch)

D Flip-Flop



- Only one input besides *ClK.*
- Implemented by a *SR* flip-flop with an additional inverter.
- When *CLK* is asserted, *Q* output follows *D* input.
- When *CLK* is not asserted, output is latched.

Characteristic Table

D	Q_t	Q_{t+1}	
0	0	0	Reset
0	1	0	
1	0	1	Set
1	1	1	

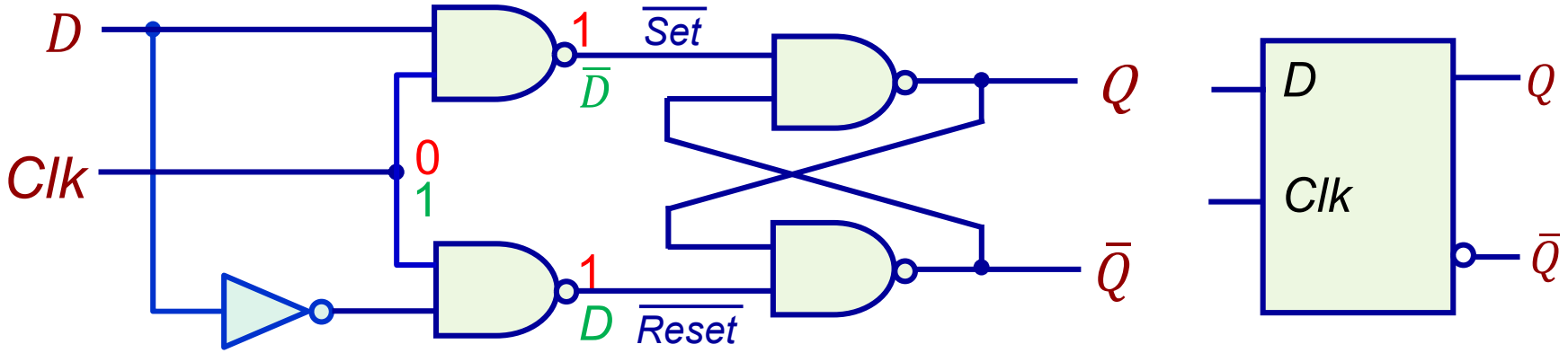
Characteristic Equation

$$Q_{t+1} = D$$

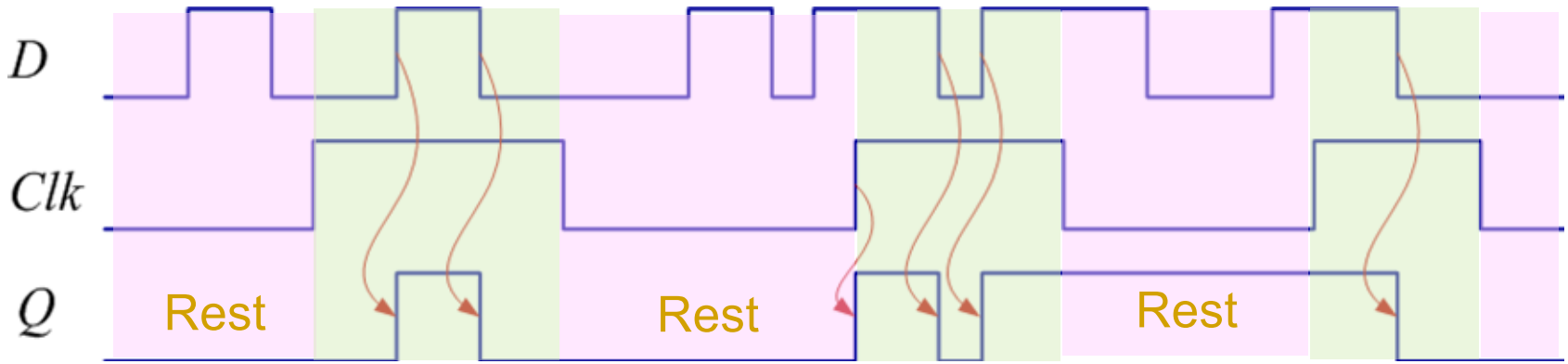
Excitation Table

$Q_t \rightarrow Q_{t+1}$	D
$0 \rightarrow 0$	0
$0 \rightarrow 1$	1
$1 \rightarrow 0$	0
$1 \rightarrow 1$	1

Timing Diagram of D Flip-Flop

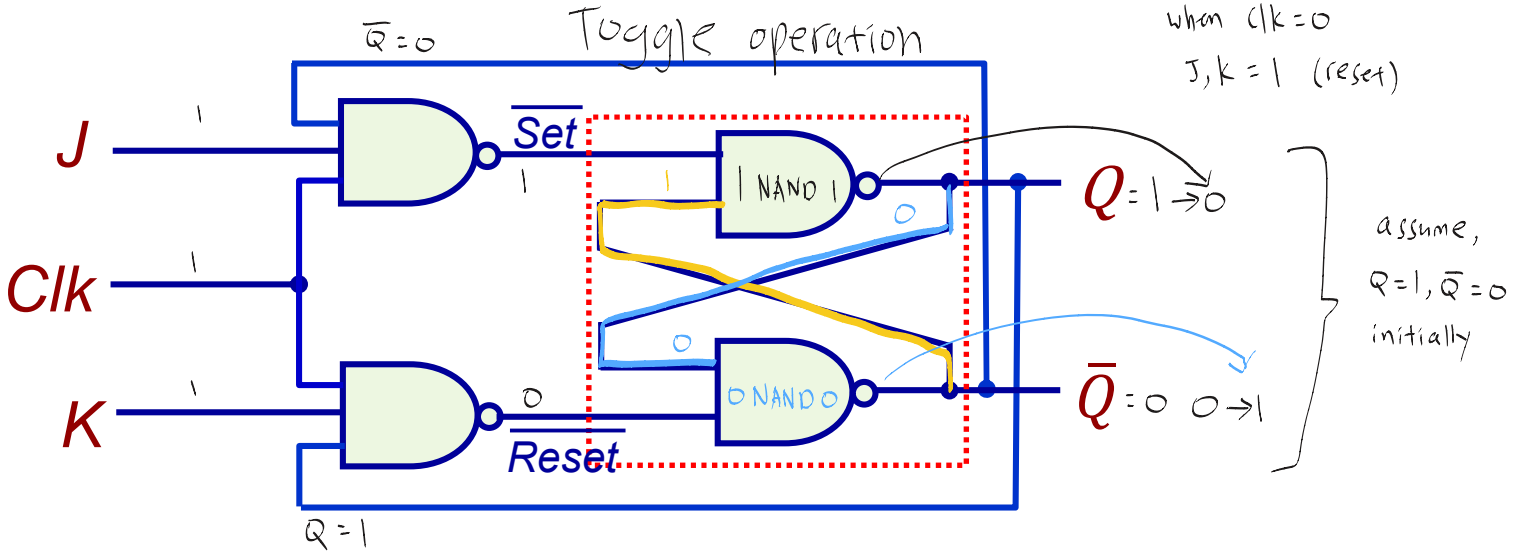


Q following D (simple)



The JK Flip-Flop

- Functionally identical to *SR* flip-flop (*J* ≡ SET, *K* ≡ RESET) except that *J* and *K* inputs are permitted to be asserted together.
- When *J* = *K* = 1, the output state toggles.
- Modified from *SR* flip-flop by direct feedback from output to the SET/RESET decoder.



Compare with the JK flip-flop designed using cross-coupled NOR cell in Tutorial 9, Q3a.

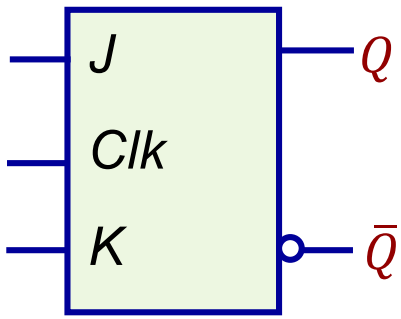
Characteristic & Excitation Tables (JK FF)

Characteristic Table

J	K	Q_t	Q_{t+1}	
0	0	0	0	Rest
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	1	Toggle
1	1	1	0	

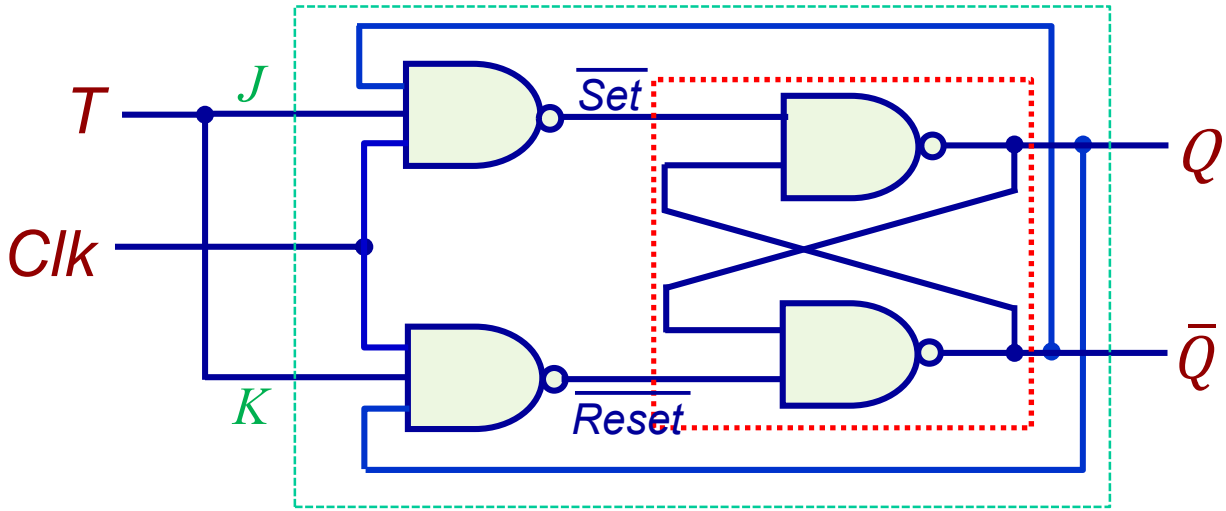
Excitation Table

Q_t	Q_{t+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0



Toggle Flip-Flop

- When the *T* input is asserted, output changes state.



Characteristic Table

T	Q_t	Q_{t+1}	
0	0	0	Rest
0	1	1	
1	0	1	Toggle
1	1	0	

Characteristic Equation

$$Q_{t+1} = \bar{T}Q_t + T\bar{Q}_t$$

$$= T \oplus Q_t$$

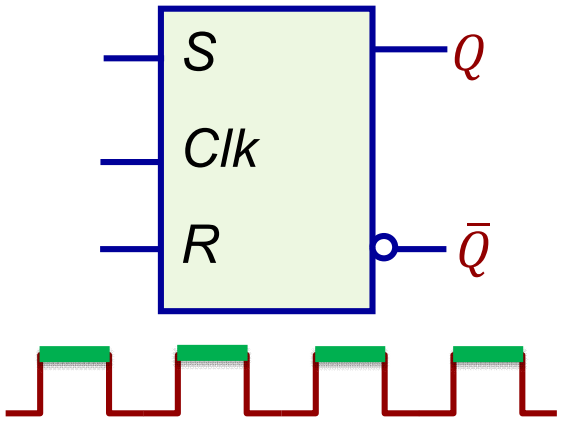
See Tutorial 8 Q5

Excitation Table

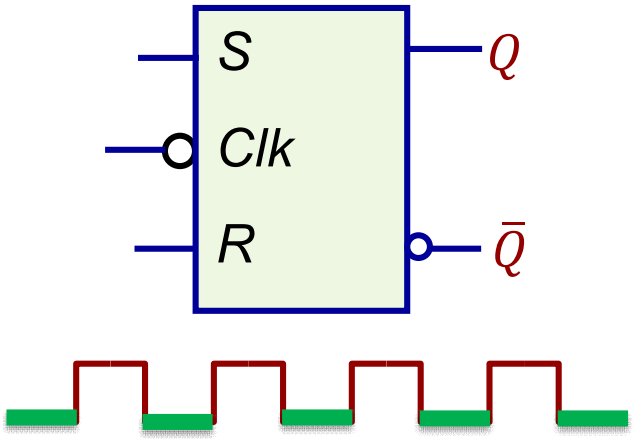
$Q_t \rightarrow Q_{t+1}$	T
$0 \rightarrow 0$	0
$0 \rightarrow 1$	1
$1 \rightarrow 0$	1
$1 \rightarrow 1$	0

FF Triggering Schemes (Symbols)

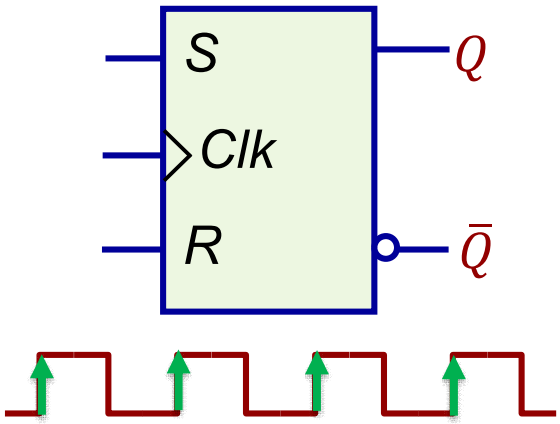
High level triggered



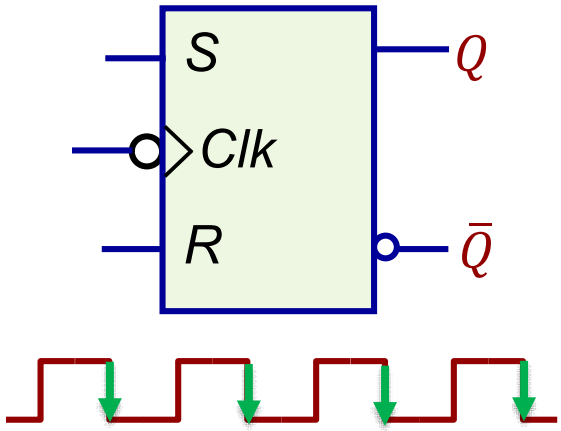
Low level triggered



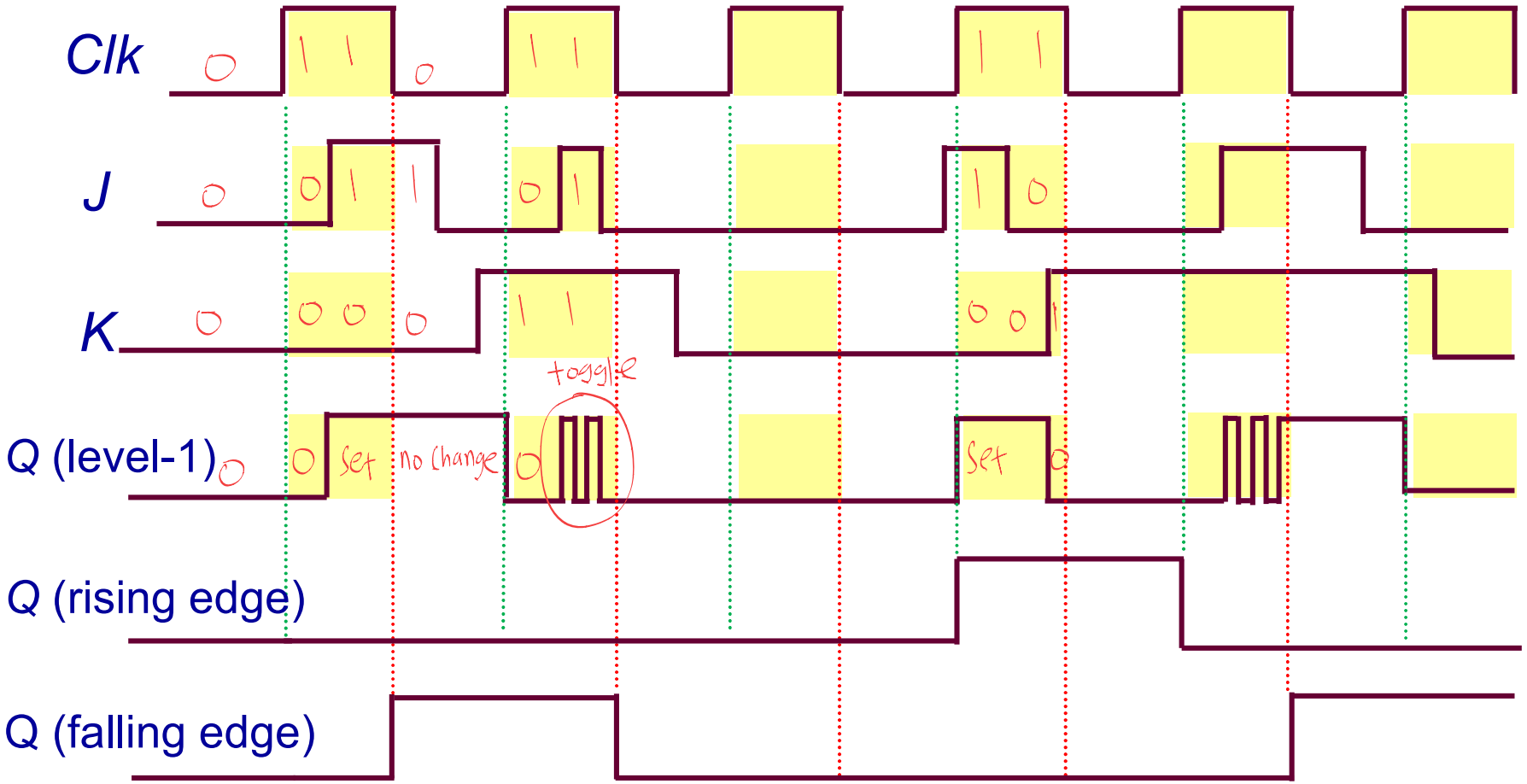
Rising edge triggered



Falling edge triggered



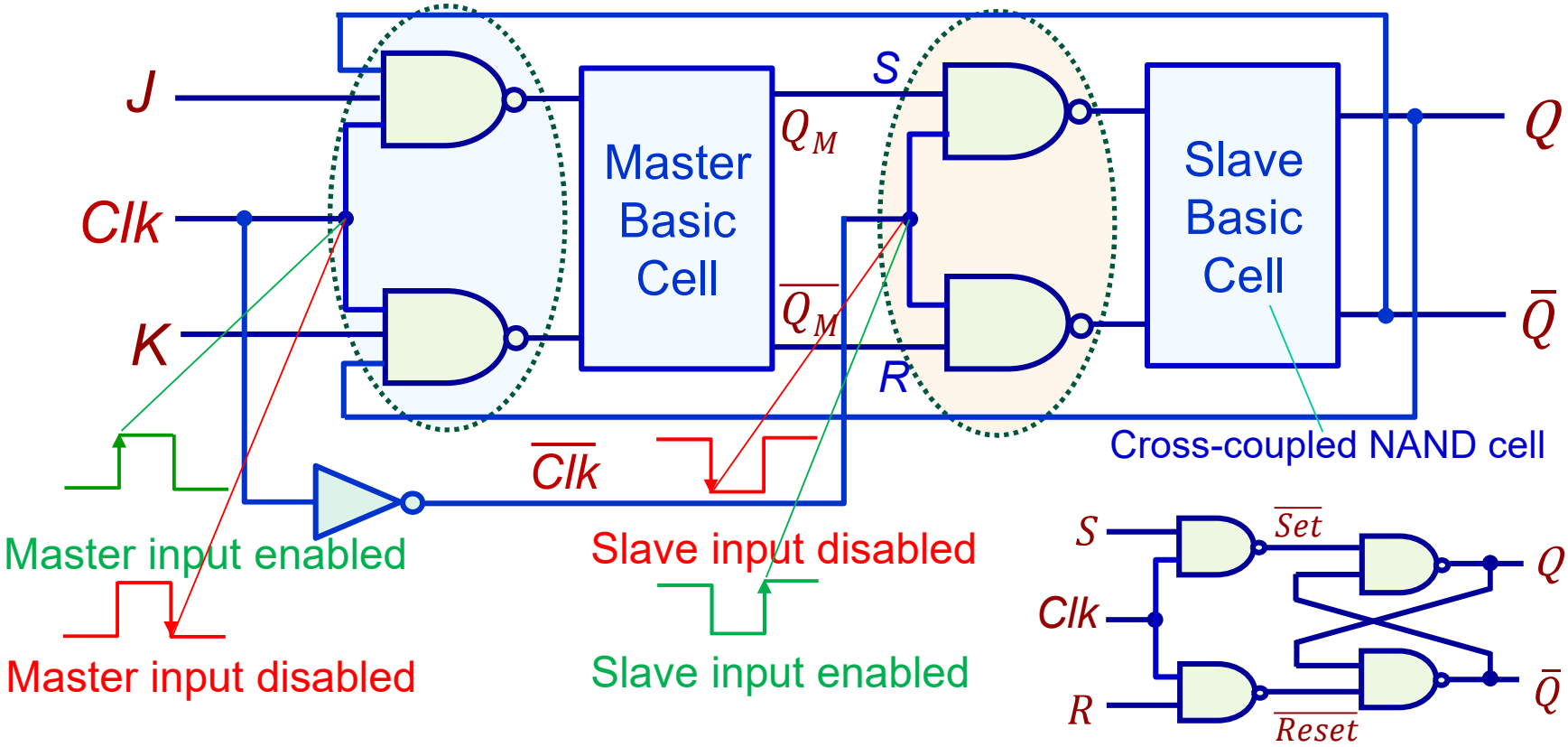
Level Triggered vs Edge Triggered



Triggering Schemes of Flip-Flops

- Level Triggering. This is normally implemented with the use of a simple gating scheme to gate the clock. However, this scheme can cause the flip-flop to be unstable under certain input conditions. The output of the flip-flop also does not necessarily change in synchrony with a clock edge.
- Pulse Triggering. This is normally implemented with the use of the master-slave system. Under normal circumstances, this system does cause the output to change with a clock edge. However, this scheme can give rise to problems known as "ones catching" and "zeros catching". (see Tutorial 9, Q4)
- Edge-Triggering. This is normally implemented with the use of the pulse transition detector. This system ensures that the output of the flip-flop changes only with a clock edge. This is now the predominant mode of triggering used in synchronous sequential systems.

Master/Slave JK Flip-Flop



The master/slave JK flip-flop is a pulse-triggered device. It guarantees stability against oscillatory state changes due to multiple operations during an asserted clock pulse (Race Problem) (see Tutorial 9, Q3).

Master/Slave JK Flip-Flop

On the rising-edge of the clock pulse:

- Master basic cell is loaded according to the input condition of the 1st SET/RESET decoder $(J, K, Q, \bar{Q})$.
- Slave basic cell is inhibited due to the inverted clock pulse is applied onto the inputs of the 2nd SET/RESET decoder.
- The new state of master basic cell is blocked by the intermediate NAND gates.

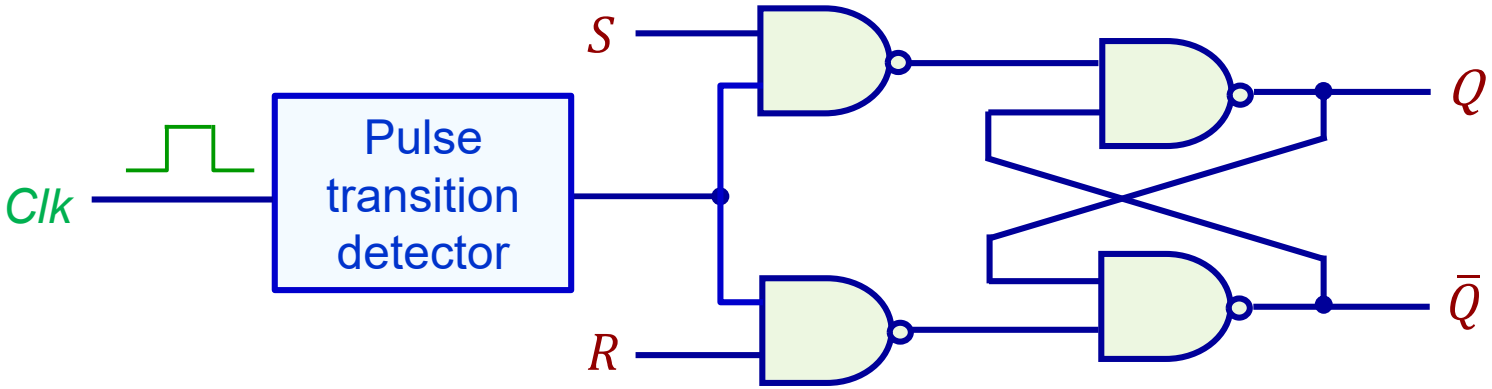
On the falling-edge of the clock pulse:

- Master cell is inhibited.
- Slave cell is enabled, data stored in the Master cell is passed through the Slave cell and the outputs , Q and $\bar{Q}$ are updated.
- The new state is fed back to the input of the 1st SET/RESET decoder.

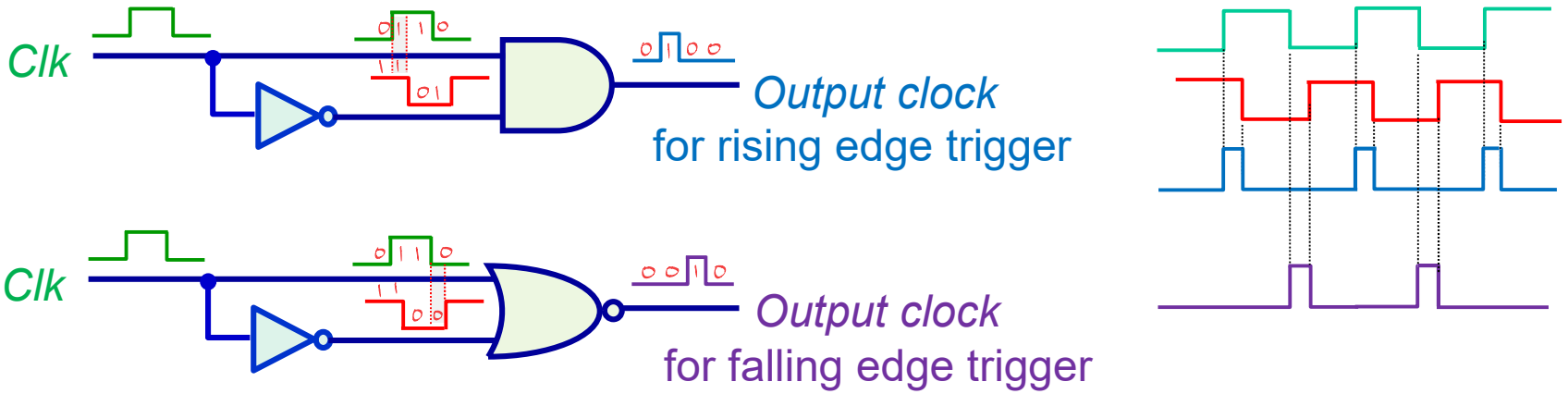
For the MASTER/SLAVE configuration in previous slide, the state changes on the falling edge of the input clock (Clk).

Edge Triggered Flop-Flop

Edge Triggering is achieved by the Pulse Transition Detector (PTD).
The PTD merely converts the CLK's rising edge to a very narrow pulse.



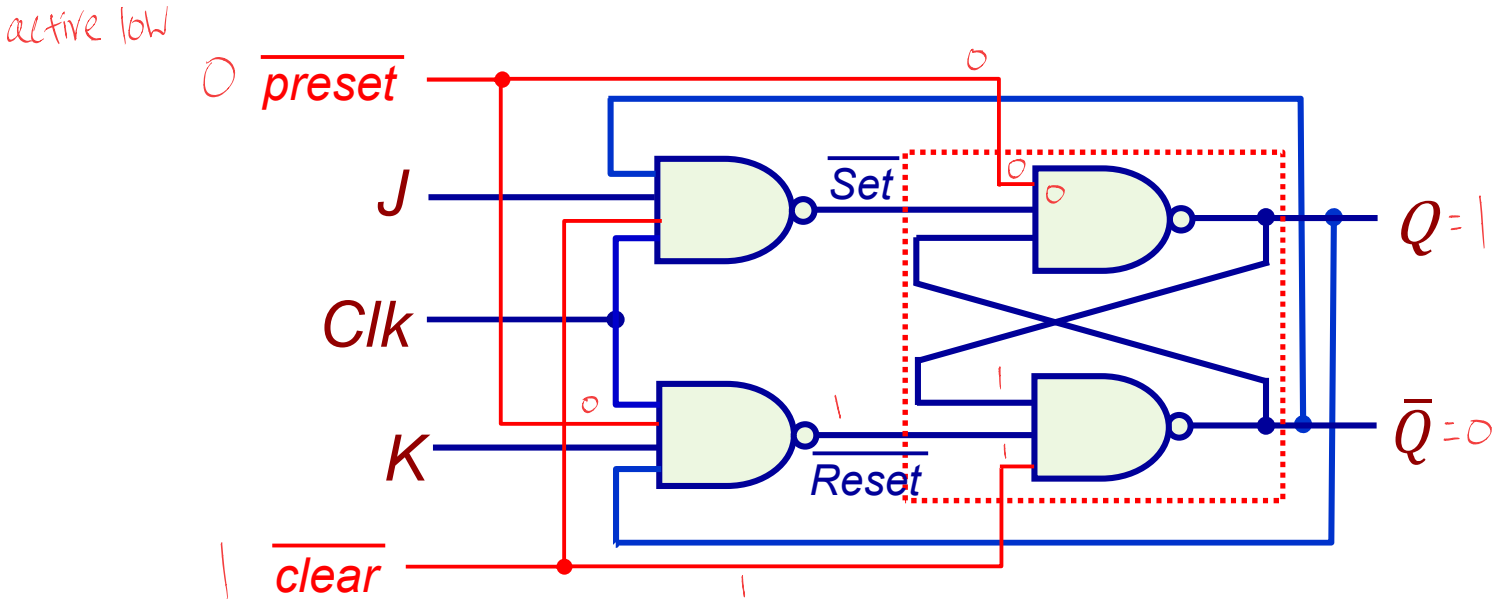
A simplified logic diagram of an edge-trigger SR flip-flop



A typical pulse transition detector (See Tutorial 9, Q4)

Asynchronous Preset and Clear

- These asynchronous inputs are standard for most integrated circuit flip-flops and are normally added in order to make the device more flexible.
- Completely overrides *J* and *K* inputs.
- These inputs are NOT synchronized by the clock signal. When asserted, they affect the output immediately.



Sequential Circuits

- In digital system, one important application is *control*, where digital signals are received and interpreted to generate control outputs according to the *sequence* in which the input signals are received.
- Combinational logic cannot satisfy this condition.
- *Sequential Logic System*, also called *Sequential Circuit, Sequential Machine*, or *State Machine,* has *memory* capability with at least one *feedback* path from memory element to the system inputs.
- A *Finite State Machine* (FSM) is a model of computation in which only one of a finite number of states can be active at one time. An FSM transits from one state to another, responding to conditions or inputs.

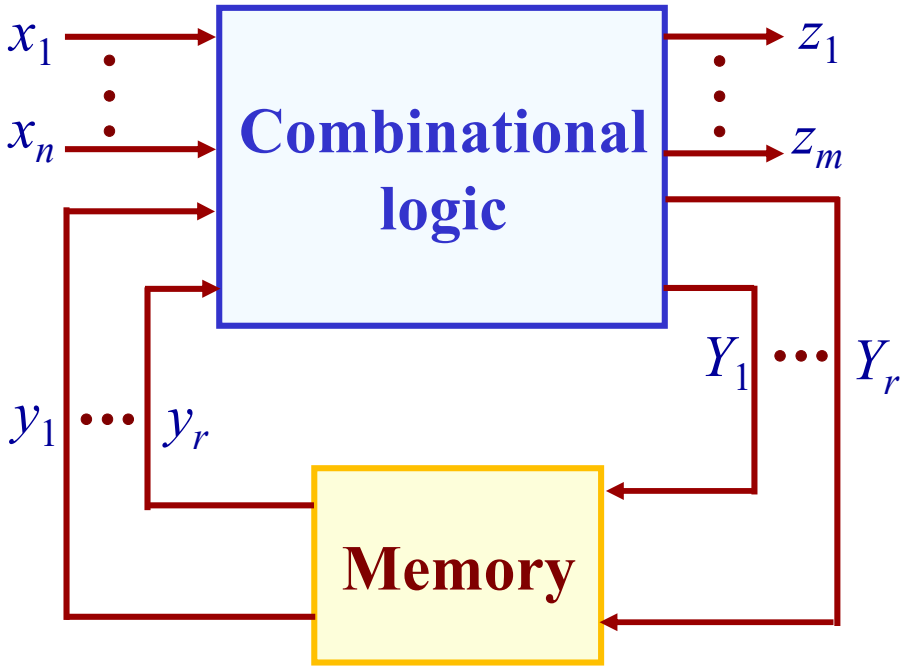
Two Types of Sequential Circuits

- *Synchronous sequential circuits* – All the states of the circuit change at the simultaneous moments. They use flip-flops for memory (to implement the states) and a synchronizing input waveform called the *clock* signal to control the timing of their operation.
- *Asynchronous sequential circuits* – The states of the circuit can change any time along with the change of input signals. There is no clock signal. They use unclocked flip-flops or time-delayed device for memory. An asynchronous sequential circuit quite often resembles a combinational circuit with feedback.
- This course focuses on synchronous sequential circuits, except for asynchronous counters.

Model of Sequential Circuits

Fundamental concepts:

- Input (x_1, x_2,…, x_n)
- Output (z_1, z_2,…, z_m)
- Present state (y_1, …, y_r)
- Next state
- State transition excitations to next state (Y_1,Y_2,…, Y_r)



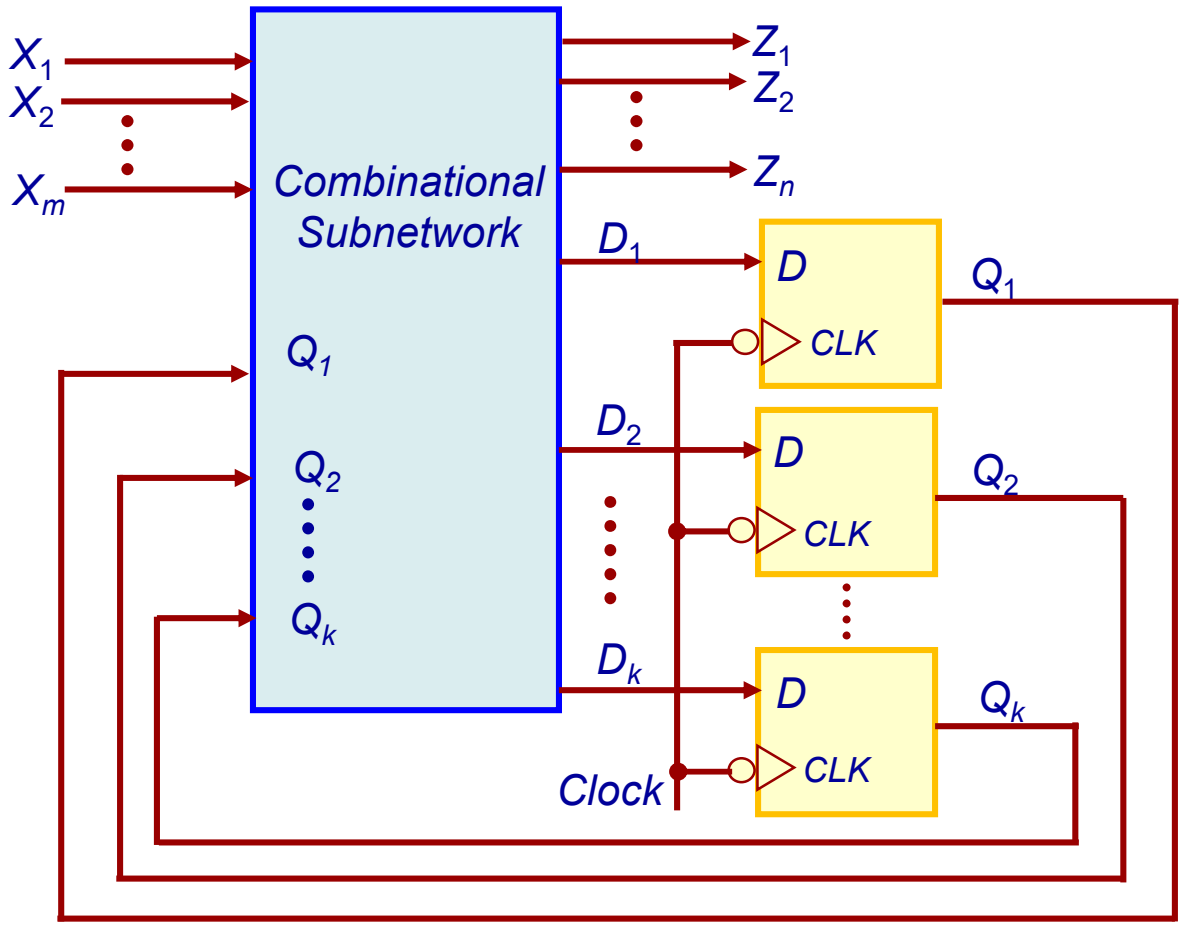
Output decoder: $$z_i = g_i\left(x_1, x_2, \cdots, x_n, y_1, y_2, \cdots, y_r\right), \quad i = 1, 2, \cdots, m$$

Next state decoder: $$Y_i = h_i\left(x_1, x_2, \cdots, x_n, y_1, y_2, \cdots, y_r\right), \quad i = 1, 2, \cdots, r$$

Note: z_i, x_i, y_i, and Y_i are all binary variables (logic 0 or 1)

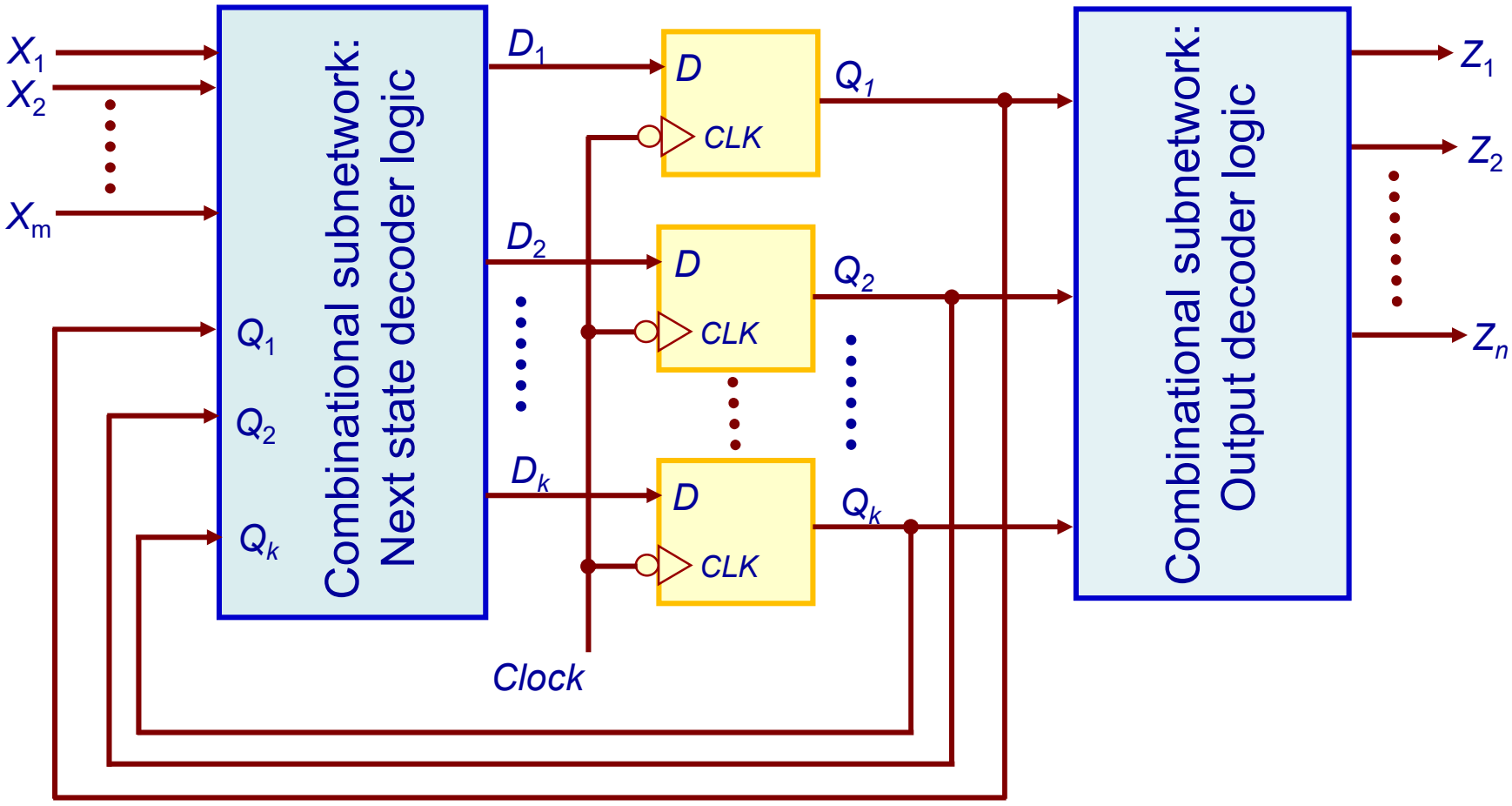
Mealy Finite State Machine (FSM)

Mealy model – Outputs depend on both external inputs and present state.



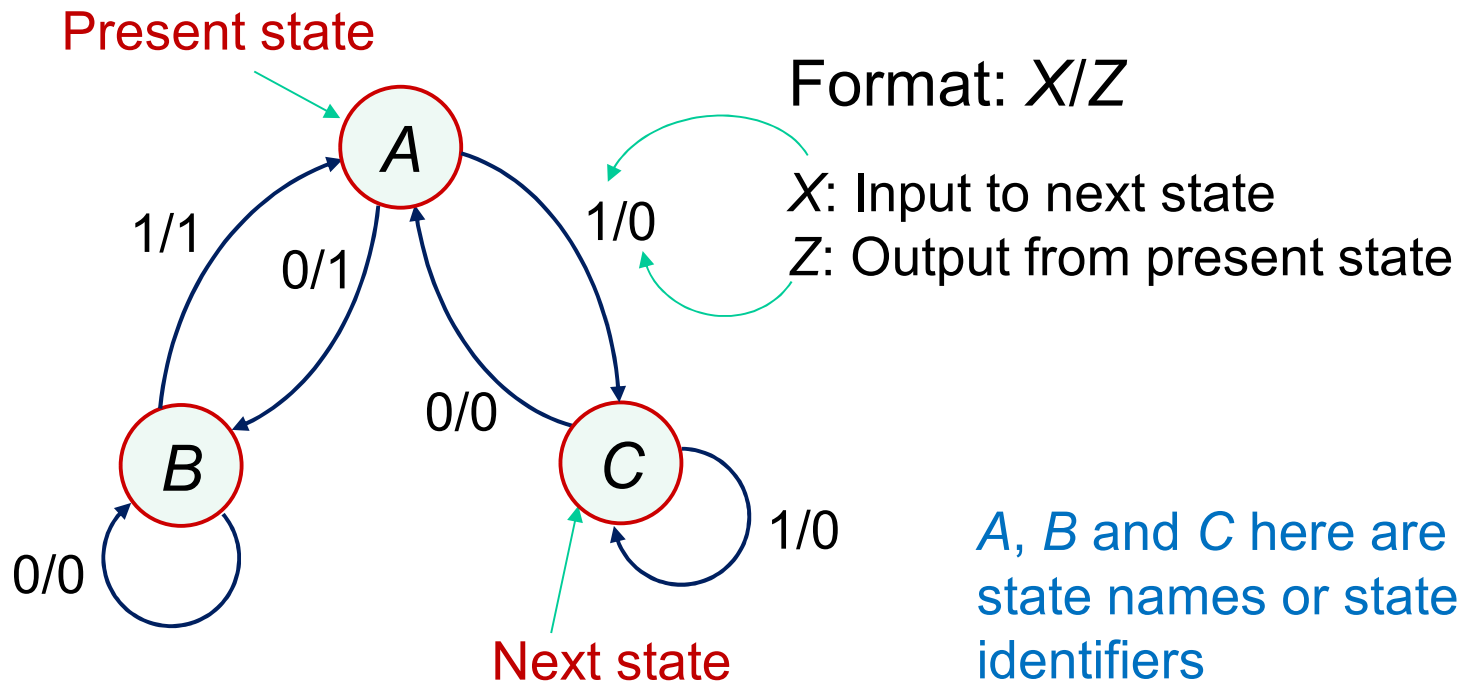
Moore Finite State Machine (FSM)

Moore model – Outputs depend ONLY on present state



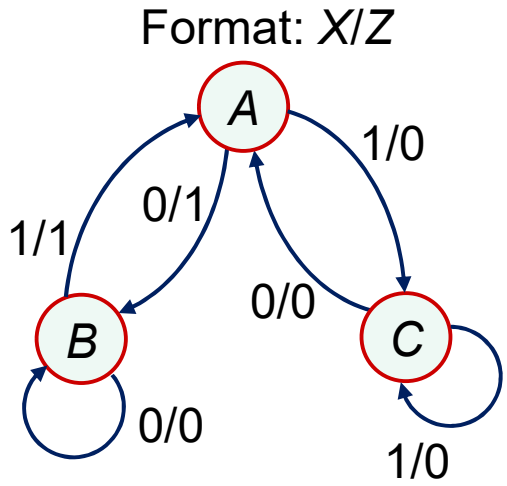
State Diagram of Mealy FSM

The operation of a sequential machine can be conveniently described by a *State Diagram*, which graphically defines the State-to-State transition of a sequential machine.



State Table of Mealy FSM

- Input variables are listed across the top.
- State identifiers are listed down the left side.
- Table entries are the next state identifier and outputs.



S	Input (X)	
	0	1
A	B, 1	C, 0
B	B, 0	A, 1
C	A, 0	C, 0

S*, Z

S	Next state (S*)		Output (Z)	
	X = 0	X = 1	X = 0	X = 1
A	B	C	1	0
B	B	A	0	1
C	A	C	0	0

How to read ?

When the circuit is in state *A*, on input *X* = 0, the circuit will proceed to the next stage *B* and output *Z* = 1.

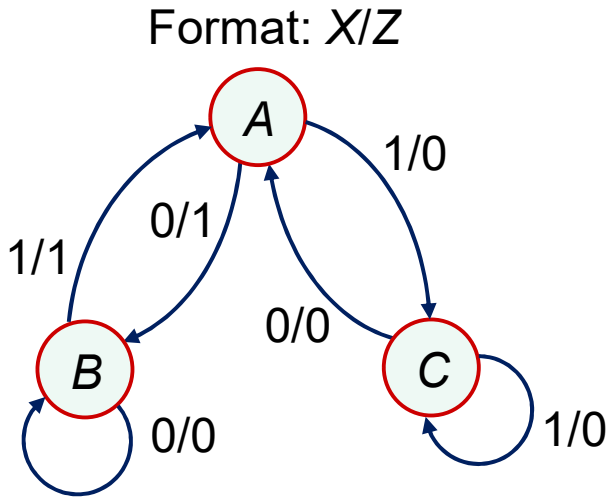
Outputs of Mealy State Machine

Determine the output response of the sequential circuit to the input sequence X = 0110 1011 00 assuming initial state is A.

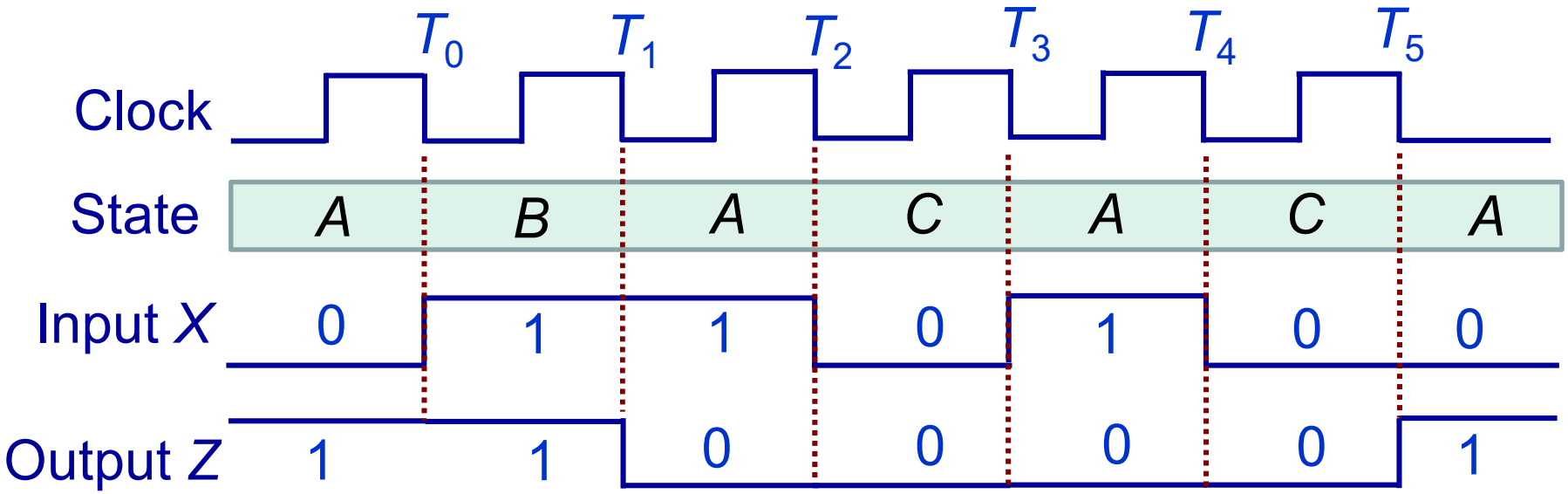
Clock Pulse:	0	1	2	3	4	5	6	7	8	9	10
Present state:	A	B	A	C	A	C	A	C	C	A	B
Input X:	0	1	1	0	1	0	1	1	0	0	
Next state:	B	A	C	A	C	A	C	C	A	B	
Output Z:	1	1	0	0	0	0	0	0	0	1	

Hence, the resulting output sequence is Z = 1100 0000 01 and the final state of the circuit is B.

What if the initial state is B or C ?



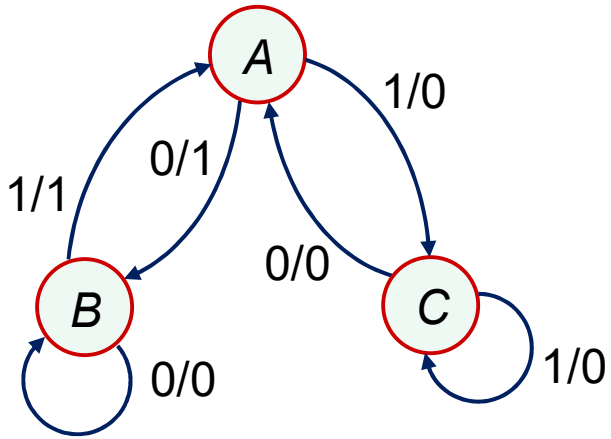
Timing Diagram of Mealy FSM (I)



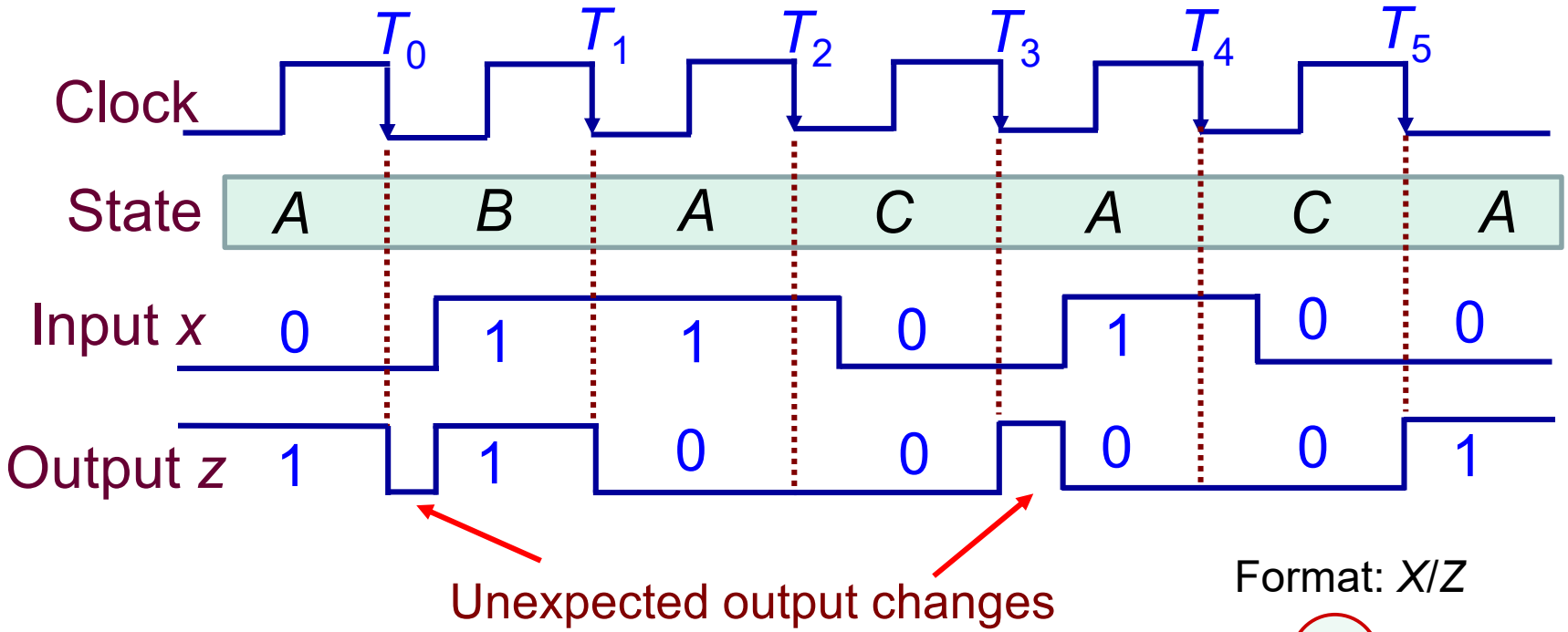
Assume falling edge trigger clock

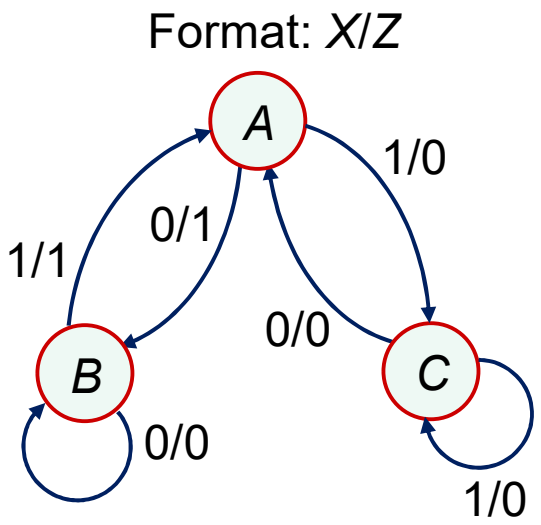
If the changes of input *X* is synchronous with the active edge of the clock, output *Z* has no spurious transitions.

Format: *X*/*Z*



Timing Diagram of Mealy FSM (II)



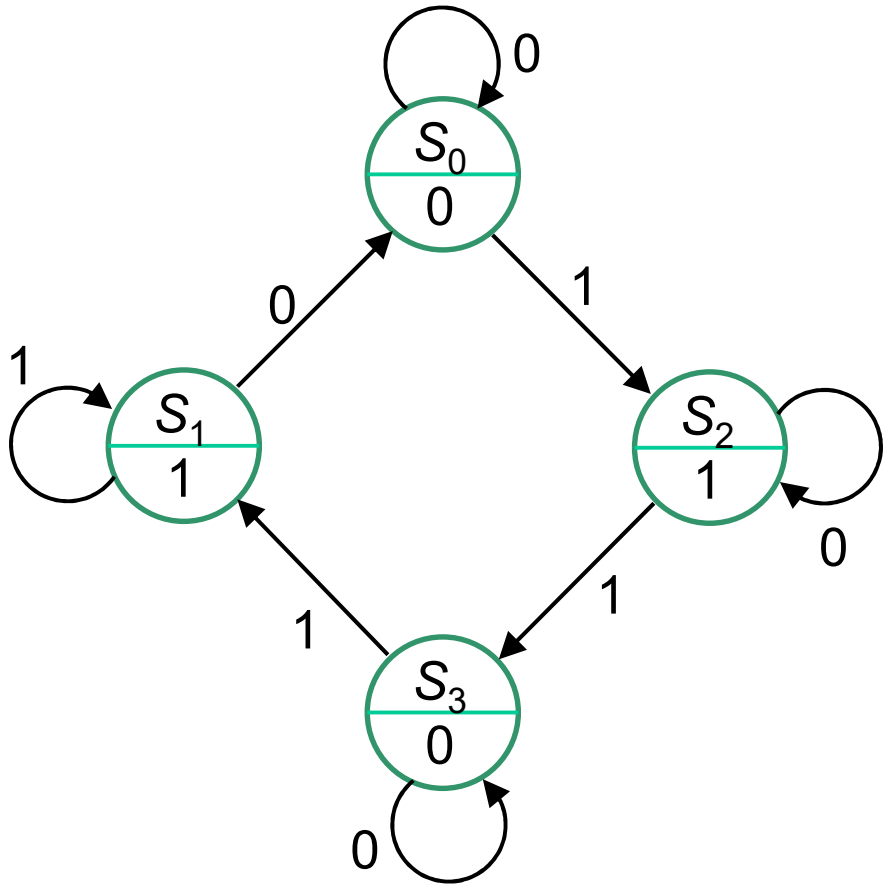


- Output z of Mealy state machine can change any time either due to input or state changes, since z is a function of both.
- The output of a Mealy state machine should be sampled only when the circuit has stabilized after an input change.

Moore FSM

State Diagram

output only change when state changes



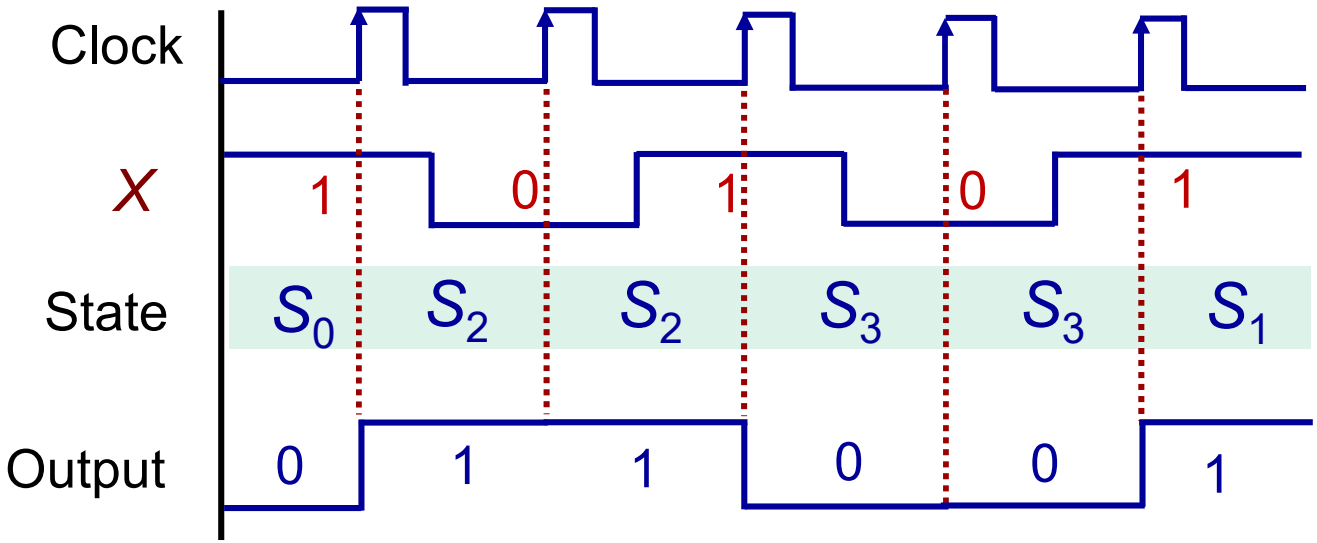
State Table

S	Input (X)		Output (Z)
	0	1	
S_0	S_0	S_2	0
S_1	S_0	S_1	1
S_2	S_2	S_3	1
S_3	S_3	S_1	0

S^*

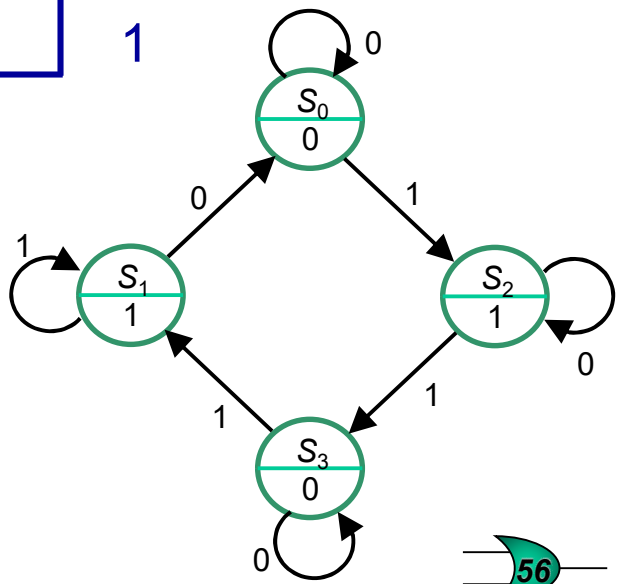
Moore FSM Timing Diagram

- Assume the initial state is A and the input sequence X = 10101. The timing diagram of the Moore FSM is:



Assume rising edge trigger clock

- Output of Moore state machine depends only on the state (represented by FFs' outputs). Therefore, the output can only change when the FFs change state.
- Input changes without state change will not cause the output to change.

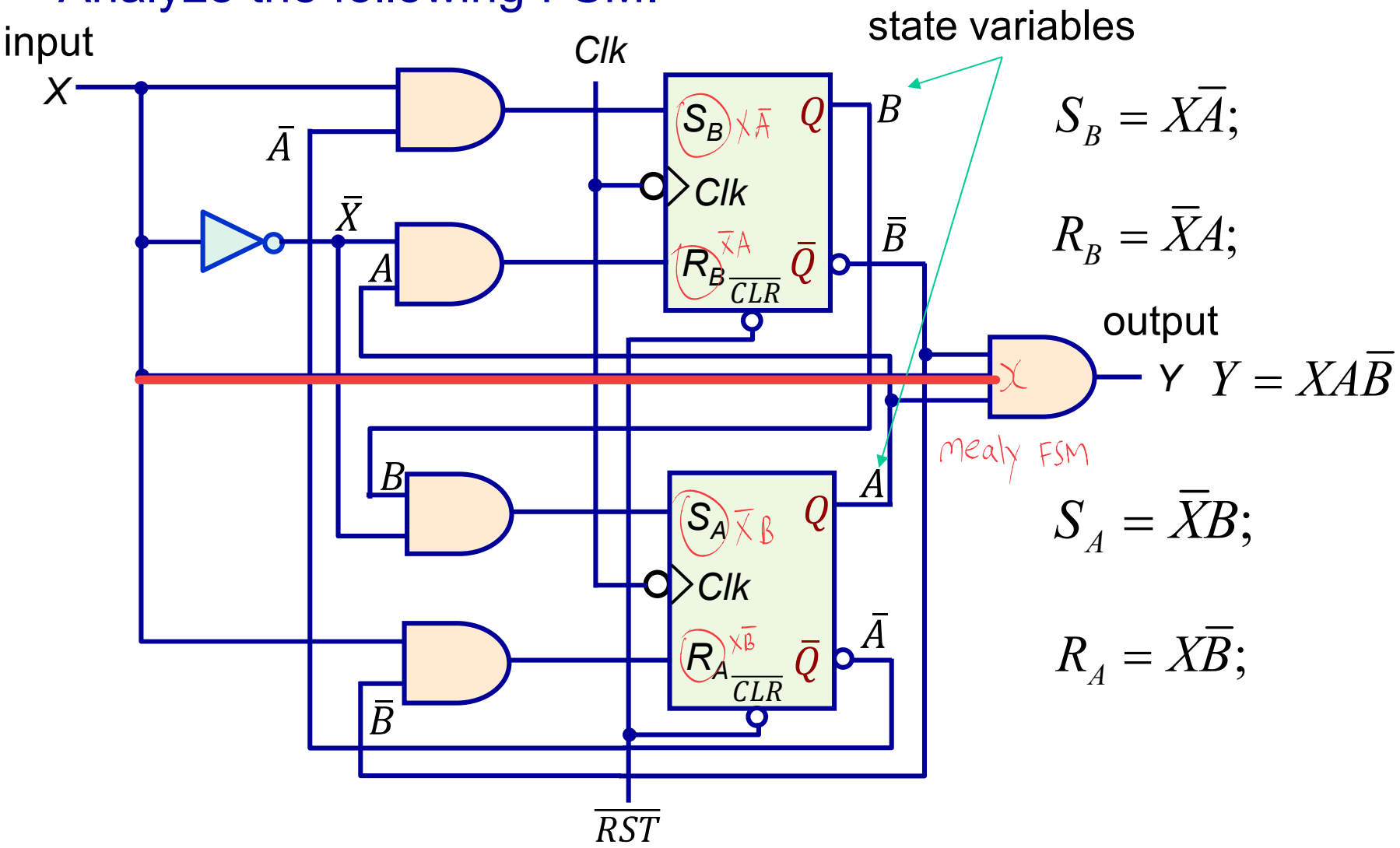


FSM Analysis

- Identify functional blocks of the FSM.
- Write *Boolean expression* for each of the outputs of *NEXT-STATE DECODER* (inputs of flip-flops), and output of the circuit.
- Plot K-maps (state map) from Boolean expressions.
- Construct *PRESENT & NEXT STATE TABLE* using K-maps (state map) and *Characteristic Table* of Flip-Flops.
- Use present state, input, output, and next state to develop a State Diagram.
- Describe behaviour based on state diagram.

Example 1

Analyze the following FSM.



Present and Next State Table

$$S_A = \bar{X}B; \quad R_A = X\bar{B}; \quad S_B = X\bar{A}; \quad R_B = \bar{X}A; \quad Y = XA\bar{B}$$

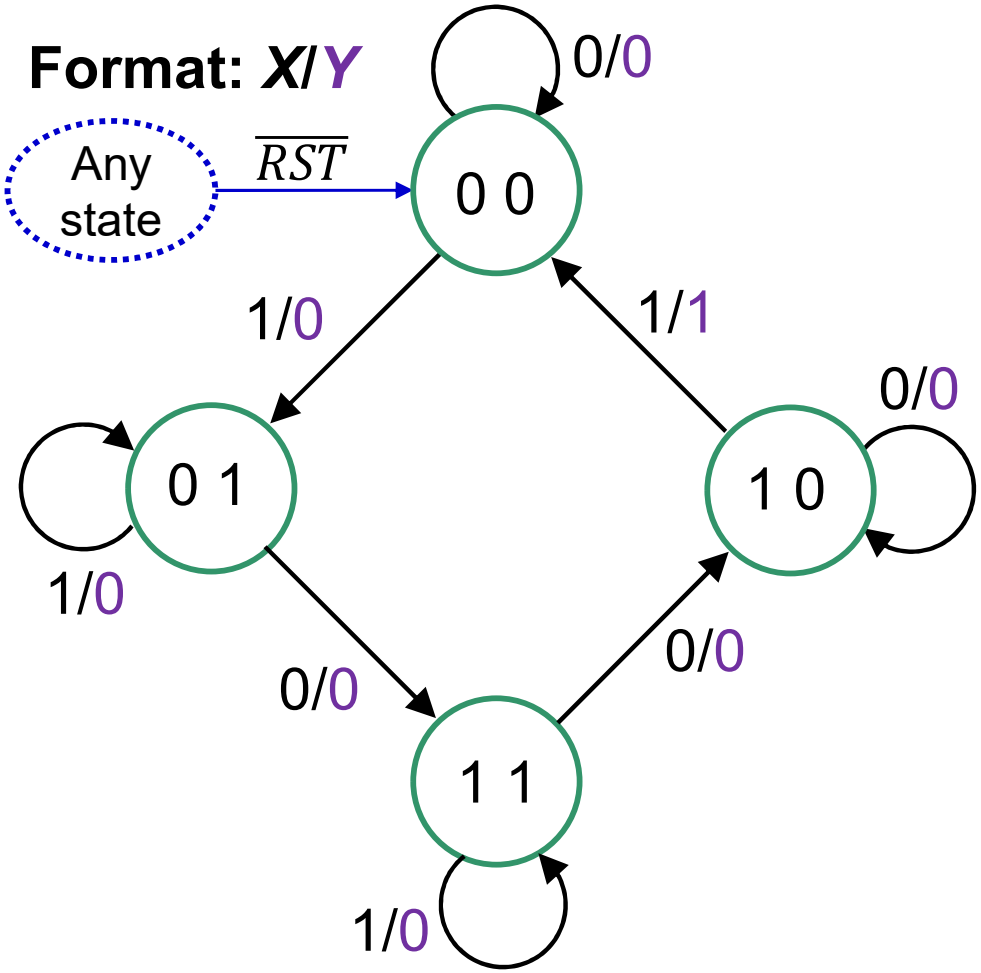
Present state		Input	Flop-Flop input				Output	Next State	
A	B	X	S_A	R_A	S_B	R_B	Y	A_{t+1}	B_{t+1}
0	0	0	0	0	0	0	0	0	0
0	0	1	0	1	1	0	0	0	1
0	1	0	1	0	0	0	0	1	1
0	1	1	0	0	1	0	0	0	1
1	0	0	0	0	0	1	0	1	0
1	0	1	0	1	0	0	1	0	0
1	1	0	1	0	0	1	0	1	0
1	1	1	0	0	0	0	0	1	1

Characteristic table

- Obtain A_{t+1} from SR next state table by comparing the A, S_A and R_A.
- Obtain B_{t+1} from SR next state table by comparing the B, S_B and R_B.

State Diagram and Analysis

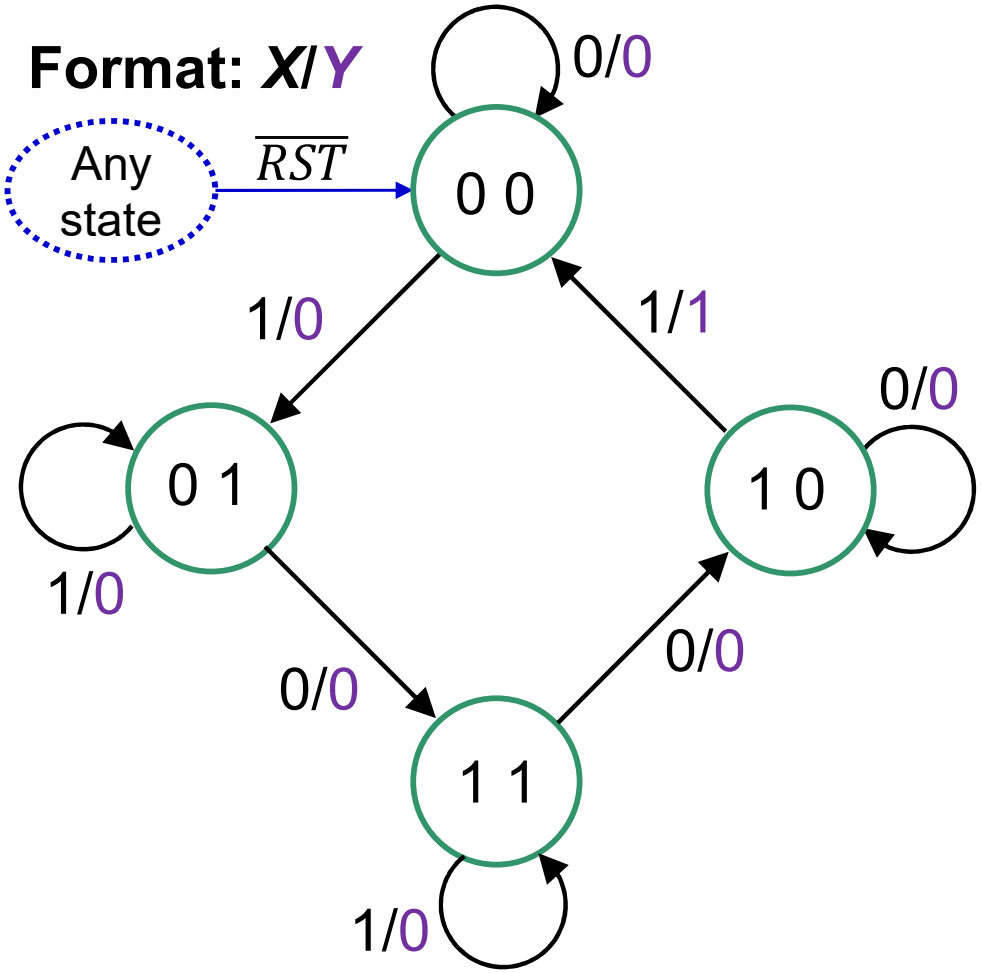
Draw state diagram and analyze its function.



A	B	X	Y	A_{t+1}	B_{t+1}
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	0	1
1	0	0	0	1	0
1	0	1	1	0	0
1	1	0	0	1	0
1	1	1	0	1	1

State Diagram and Analysis

Draw state diagram and analyze its function.

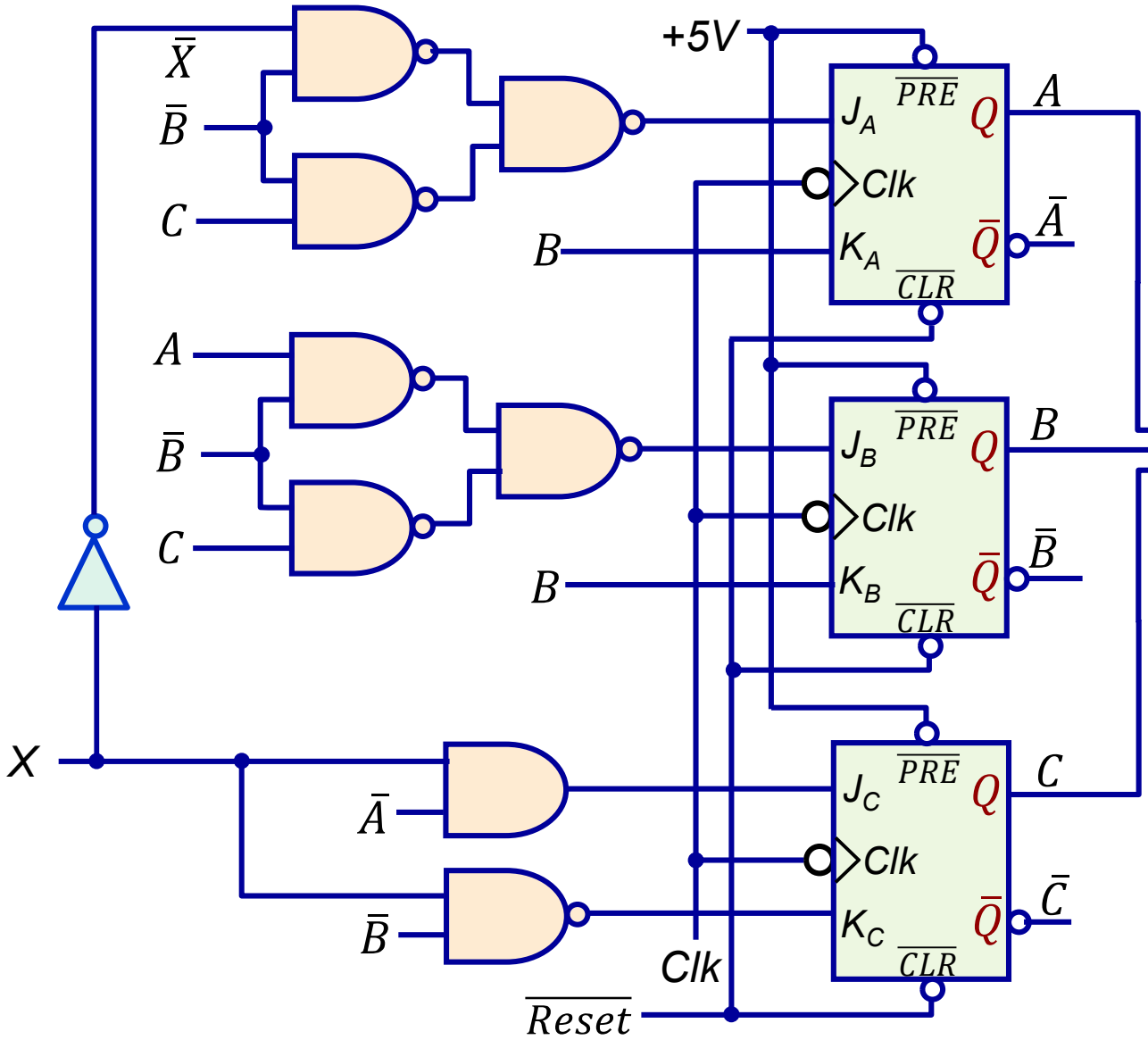


Analysis:

- It leaves reset state (*AB* = "00") when input *X* = 1.
- From state "01", It needs at least two '0's in *X* (needs not be consecutive) to reach state "10".
- In state "10", it returns to reset state and outputs '1' when the input *X* = 1.
- What is the function of this sequential circuit?

It detects any length of (non-overlapping) input string pattern that starts with '1' and end with '1' with at least two zeros between them.

Example 2: FSM Analysis



$$J_A = \bar{B}\bar{X} + \bar{B}C$$

$$K_A = B$$

$$LOAD = ABCX$$

$$J_B = A\bar{B} + \bar{B}C$$

$$K_B = B$$

$$J_C = \bar{A}X$$

$$K_C = \overline{X\bar{B}}$$

Next State and Output Equations

- Derive the logic expressions for the inputs of the flip-flops (next state decoders) and the output of the FSM.
- Draw the next state and output K-maps.

$$J_A = \overline{B}C + B\overline{X}$$

CX \ AB	00	01	11	10
00	1			1
01				
11	1			1
10	1			1

$$J_B = A\overline{B} + \overline{B}C$$

CX \ AB	00	01	11	10
00				1
01				1
11	1			1
10	1			1

$$J_C = \overline{A}X$$

CX \ AB	00	01	11	10
00				
01	1	1		
11	1	1		
10				

$$K_A = K_B = B$$

CX \ AB	00	01	11	10
00		1	1	
01		1	1	
11		1	1	
10		1	1	

$$K_C = \overline{X\overline{B}} = \overline{X} + B$$

CX \ AB	00	01	11	10
00	1	1	1	1
01		1	1	
11		1	1	
10	1	1	1	1

$$LOAD = ABCX$$

CX \ AB	00	01	11	10
00				
01				
11			1	
10				

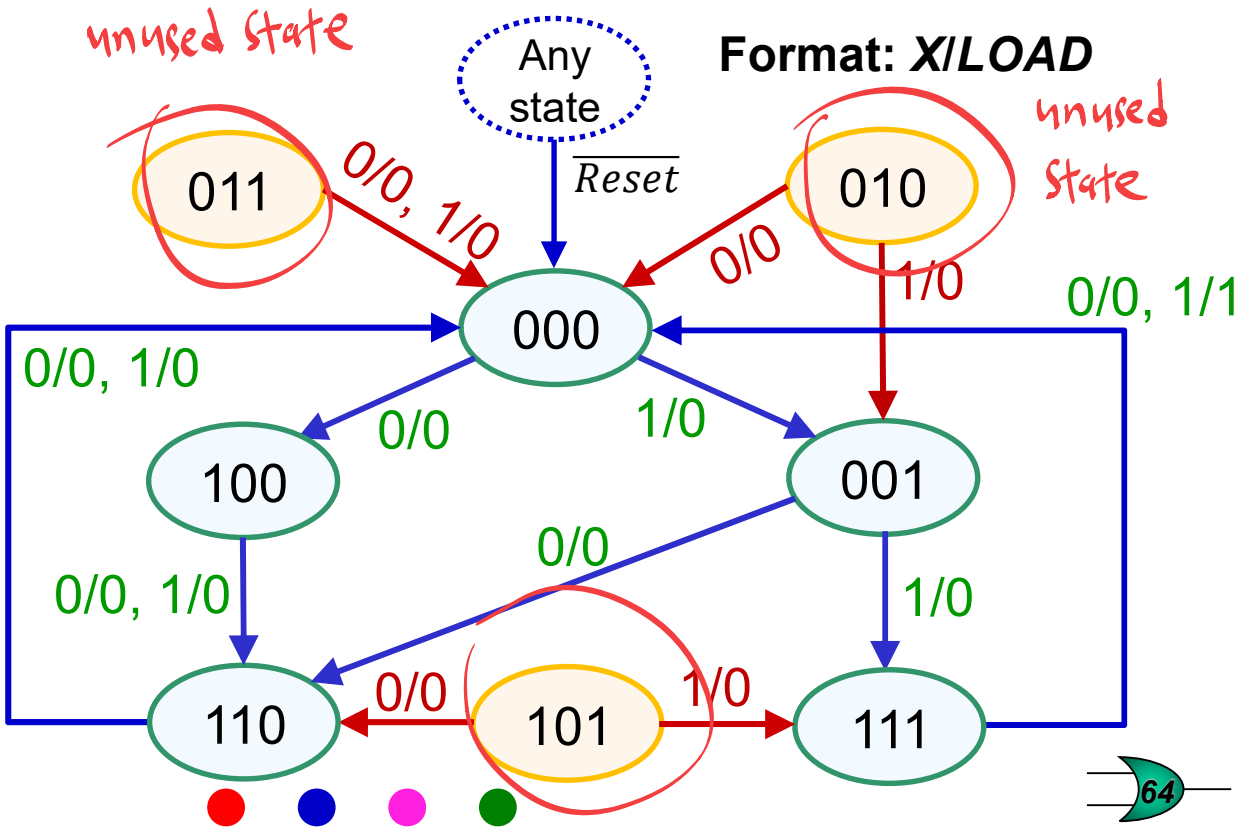
Present and Next State Table

Present state			I/p	Flop-Flop input						O/p	Next State		
A	B	C	X	J_A	K_A	J_B	K_B	J_C	K_C	Load	A^*	B^*	C^*
0	0	0	0	1	0	0	0	0	1	0	1	0	0
0	0	0	1	0	0	0	0	1	0	0	0	0	1
0	0	1	0	1	0	1	0	0	1	0	1	1	0
0	0	1	1	1	0	1	0	1	0	0	1	1	1
0	1	0	0	0	1	0	1	0	1	0	0	0	0
0	1	0	1	0	1	0	1	1	1	0	0	0	1
0	1	1	0	0	1	0	1	0	1	0	0	0	0
0	1	1	1	0	1	0	1	1	1	0	0	0	0
1	0	0	0	1	0	1	0	0	1	0	1	1	0
1	0	0	1	0	0	1	0	0	0	0	1	1	0
1	0	1	0	1	0	1	0	0	1	0	1	1	0
1	0	1	1	1	0	1	0	0	0	0	1	1	1
1	1	0	0	0	1	0	1	0	1	0	0	0	0
1	1	0	1	0	1	0	1	0	1	0	0	0	0
1	1	1	0	0	1	0	1	0	1	0	0	0	0
1	1	1	1	0	1	0	1	0	1	1	0	0	0

State Diagram and Analysis

- States 000, 001, 100, 110 and 111 are sequential states.
- States 010, 011 and 101 are never reached unless NOISE or a POWER ON CONDITION forces the circuit into these states. They are unused states.
- Unwanted states that hold the machine must be avoided. They are HOLD states (unused).
- The above 3 states are not HOLD states - will not stay for more than one clock cycle.

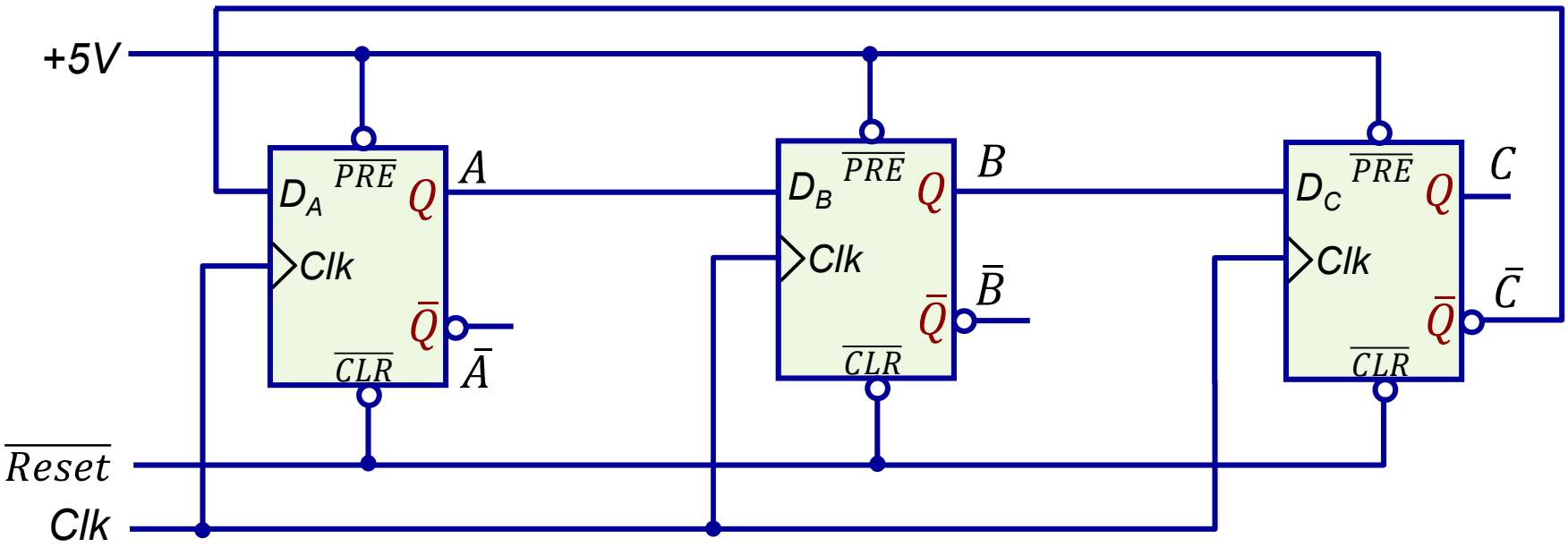
PS			I/p	O/p	NS		
A	*B*	*C*	*X*	*Load*	*A**	*B**	*C**
0	0	0	0	0	1	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	1	1	0
0	0	1	1	0	1	1	1
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	1	1	0
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	0
1	0	1	1	0	1	1	1
1	1	0	0	0	0	0	0
1	1	0	1	0	0	0	0
1	1	1	0	0	0	0	0
1	1	1	1	1	0	0	0



Programmable Hold States

- Programmable Hold States are states in which a circuit can be placed with a specific input condition. The system will hold, and no input condition except for a special reserved input will release or reset the system.
- This temporary holding operation can be a useful debugging tool. The debug feature can be built into a software or computer program.

Example 3: Twisted Ring Counter



$$D_A = \bar{C} \qquad D_B = A \qquad D_C = B$$

3 FFs $\Rightarrow$ state variables, A, B and C

No external input, no external output except FF outputs

Flip-Flop Equations (Inputs of FFs)

$$D_A = \overline{C}$$

C \ AB	00	01	11	10
0	1	1	1	1
1	0	0	0	0

$$D_B = A$$

C \ AB	00	01	11	10
0	0	0	1	1
1	0	0	1	1

$$D_C = B$$

C \ AB	00	01	11	10
0	0	1	1	0
1	0	1	1	0

C \ AB	00	01	11	10
0	100	101	111	110
1	000	001	011	010

PRESENT and NEXT STATE map

Present and Next State Table

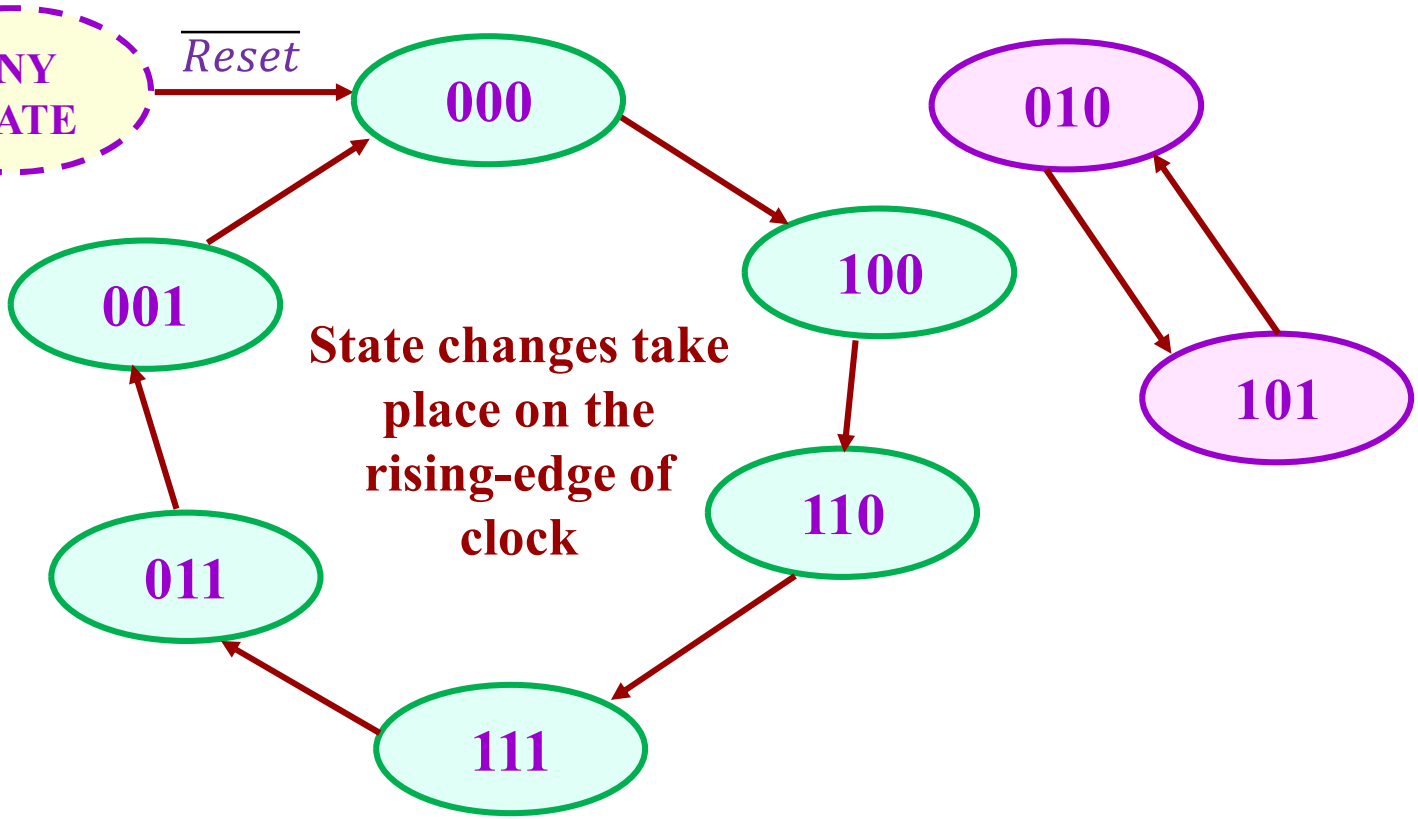
$$D_A = \overline{C};\ D_B = A;\ D_C = B$$

Present state			Flop-Flop input			Next State		
A	B	C	D_A	D_B	D_C	A^*	B^*	C^*
0	0	0	1	0	0	1	0	0
0	0	1	0	0	0	0	0	0
0	1	0	1	0	1	1	0	1
0	1	1	0	0	1	0	0	1
1	0	0	1	1	0	1	1	0
1	0	1	0	1	0	0	1	0
1	1	0	1	1	1	1	1	1
1	1	1	0	1	1	0	1	1

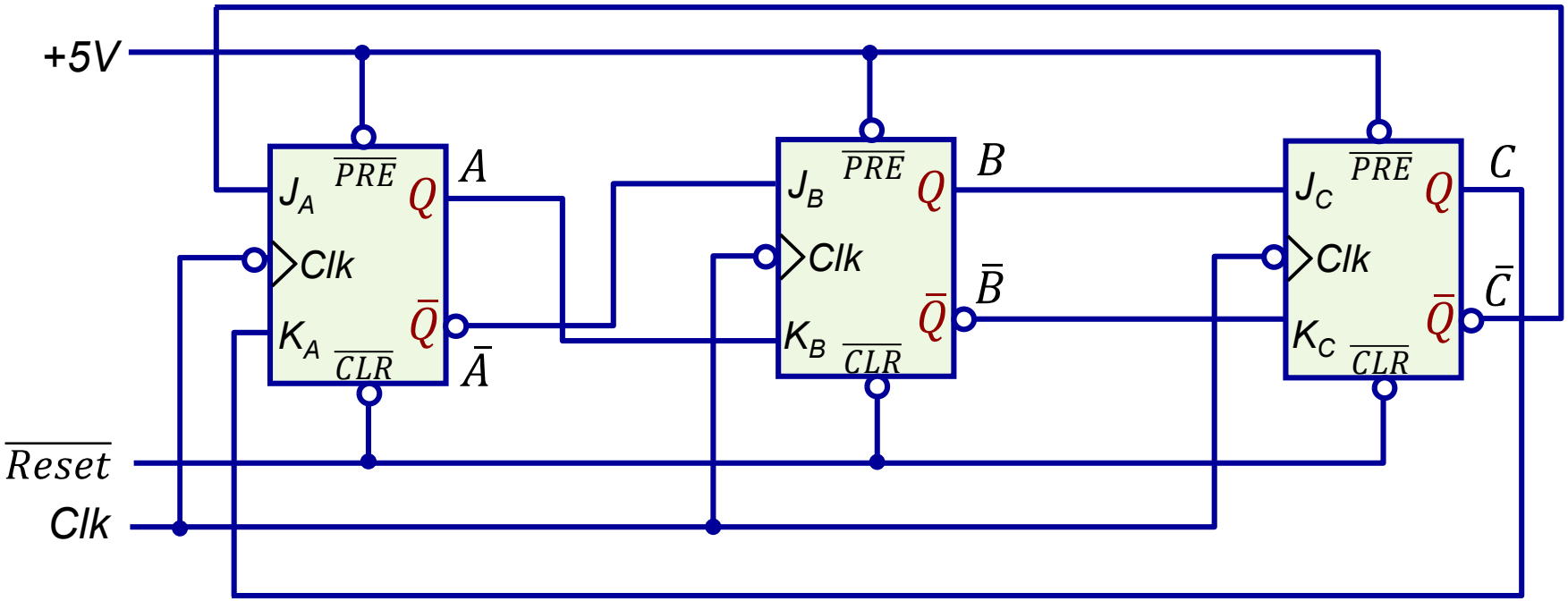
State Diagram and Analysis

- Two Hold states (010 and 101) are isolated from the main sequence states.
- This is typical of a "twisted-ring counter" circuit.

PS			NS		
A	*B*	*C*	*A**	*B**	*C**
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	0	1	0
1	1	0	1	1	1
1	1	1	0	1	1



Example 4: FSM Analysis



$$J_A = \bar{C} \qquad J_B = \bar{A} \qquad J_C = B$$

$$K_A = C \qquad K_B = A \qquad K_C = \bar{B}$$

3 FFs $\Rightarrow$ state variables, A, B and C

No external input, no external output except FF outputs

Next State Maps (FF Equations)

$J_A = \overline{C}$

C \ AB	00	01	11	10
0	1	1	1	1
1	0	0	0	0

$K_A = C$

C \ AB	00	01	11	10
0	0	0	0	0
1	1	1	1	1

$J_B = \overline{A}$

C \ AB	00	01	11	10
0	1	1	0	0
1	1	1	0	0

$K_B = A$

C \ AB	00	01	11	10
0	0	0	1	1
1	0	0	1	1

$J_C = B$

C \ AB	00	01	11	10
0	0	1	1	0
1	0	1	1	0

$K_C = \overline{B}$

C \ AB	00	01	11	10
0	1	0	0	1
1	1	0	0	1

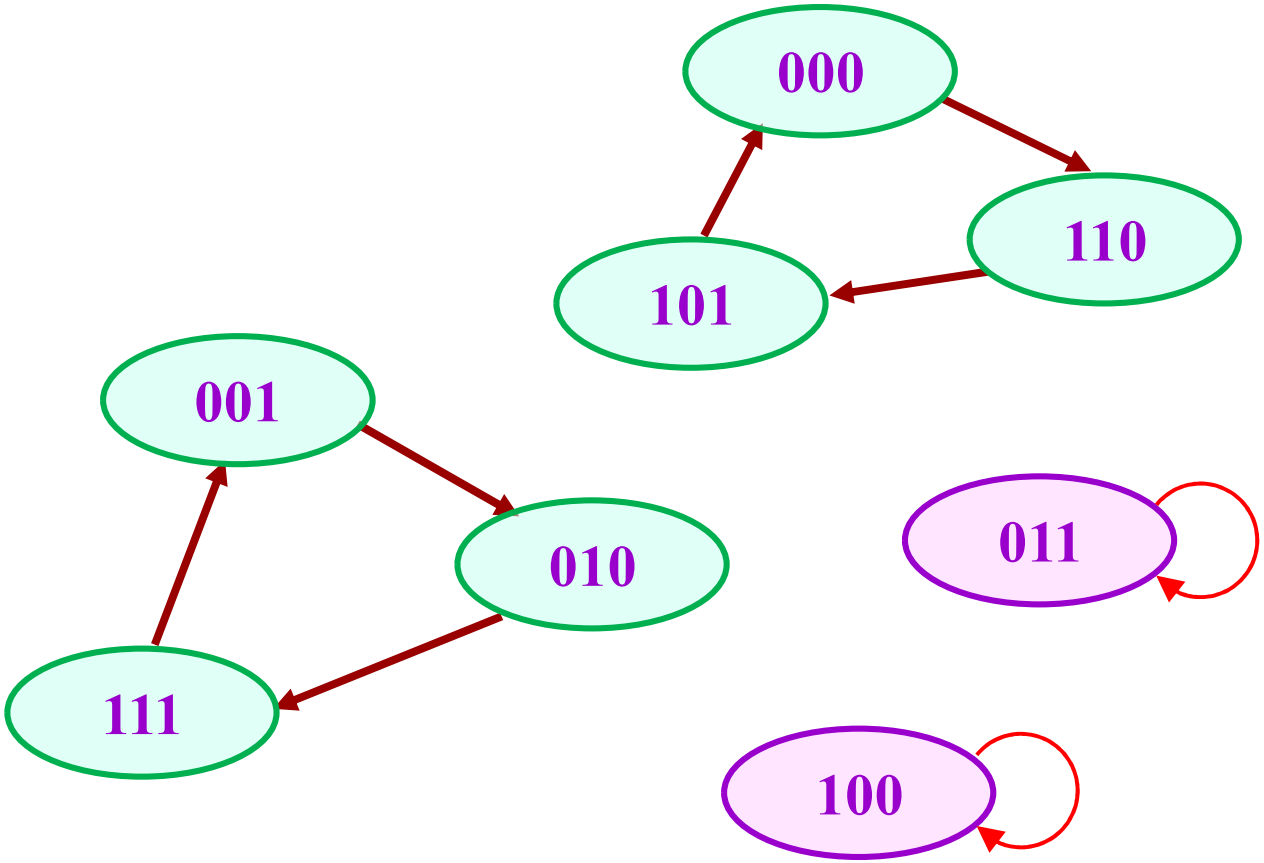
Present and Next State Table

$J_A = \overline{C}, K_A = C;$ $J_B = \overline{A}, K_B = A;$ $J_C = B, K_C = \overline{B};$

Present state			Flop-Flop input						Next State		
A	B	C	J_A	K_A	J_B	K_B	J_C	K_C	A^*	B^*	C^*
0	0	0	1	0	1	0	0	1	1	1	0
0	0	1	0	1	1	0	0	1	0	1	0
0	1	0	1	0	1	0	1	0	1	1	1
0	1	1	0	1	1	0	1	0	0	1	1
1	0	0	1	0	0	1	0	1	1	0	0
1	0	1	0	1	0	1	0	1	0	0	0
1	1	0	1	0	0	1	1	0	1	0	1
1	1	1	0	1	0	1	1	0	0	0	1

State Diagram

PS			NS		
A	*B*	*C*	*A**	*B**	*C**
0	0	0	1	1	0
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1



Design of Synchronous FSM

- Study circuit specifications.
- Draw a block diagram, and/or timing diagram, and/or flow chart, and identify the inputs and outputs.
- Draw State Diagram.
- Develop Present State/Next State Table.
- Determine the number of flip-flops. The number of flip-flops dictates the number of state variables.
- Assign binary values to each state. The binary values are called *state assignment codes*.
- Decide on flip-flop type to construct *characteristic truth-table*.
- Use K-maps to derived logic equations for Next State Decoder and Output Decoder.
- Draw sequential circuit based on the logic equations.

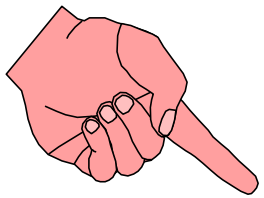
State Diagram Construction Guidelines

- Understand the problem statement by constructing sample input and output sequences.
- Determine the reset conditions.
- If only a few sequences lead to a nonzero output, it is good to start constructing partial state diagram with those sequences. Another way is to set up states for sequences or groups of sequences that must be “remembered” by the network.
- When adding arrow, determine if it can go to one of the previously defined states before creating a new state.
- Verify that for each combination of values of the input variables, there is one and only one path leaving each state.
- Test if the correct output sequences can be obtained from your diagram with the sample input sequences.

Design A Sequence Recognizer

Design a circuit with one input X and one output Z that recognizes the input sequence **101**. The circuit is also required to recognize the overlapping sequences.

For example:

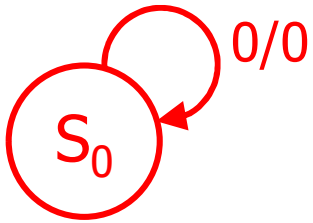


Overlapping 101 sequence

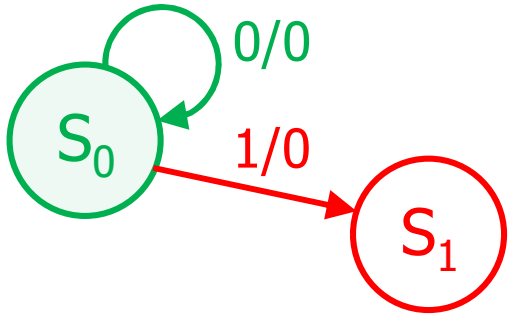
X =	0	0	1	1	0	1	1	0	0	1	0	1	0	1	0	0
Z =	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0

State Diagram

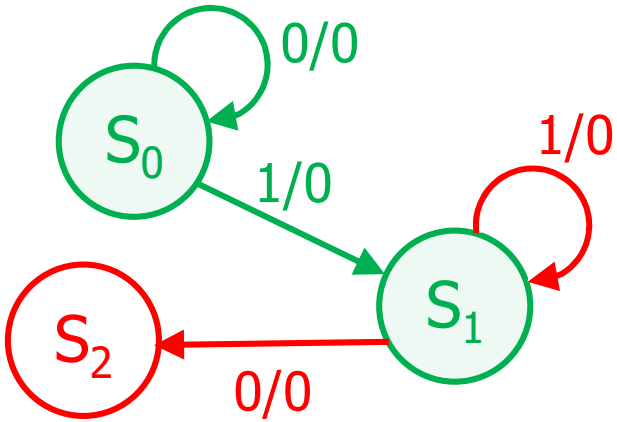
Start with reset state

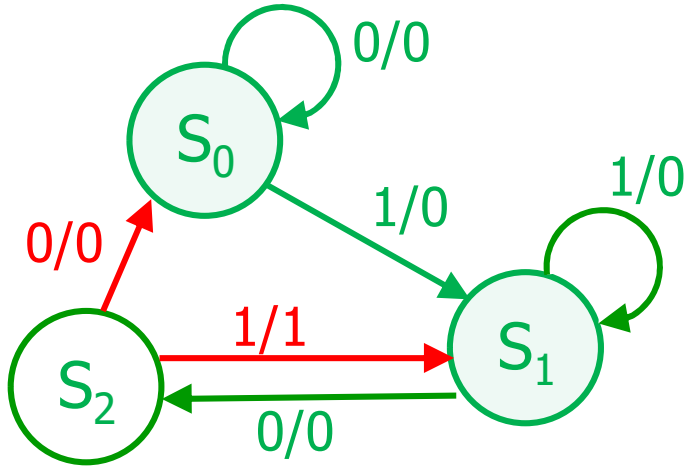


If a 1 is received, go to new state S_1 - to "remember" that the first input in the desired sequence has been received.



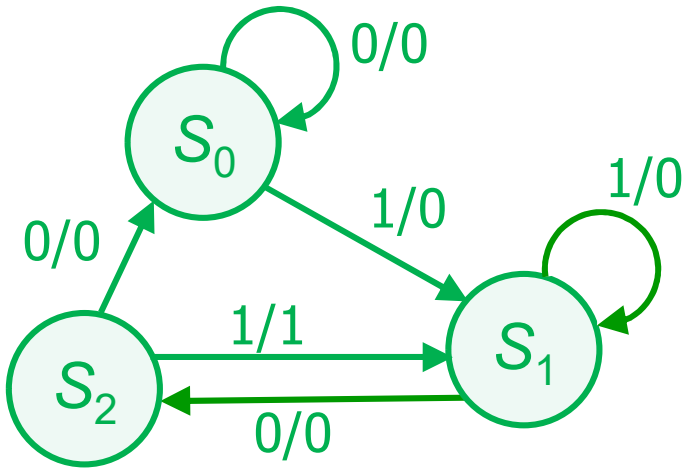
S_2 created to "remember" that the first two inputs in the desired sequence (10) have been received.





Number of State Variables

Format: *X*/*Z*



Let n_S be the number of states and n_F the number of flip-flops (state variables), then

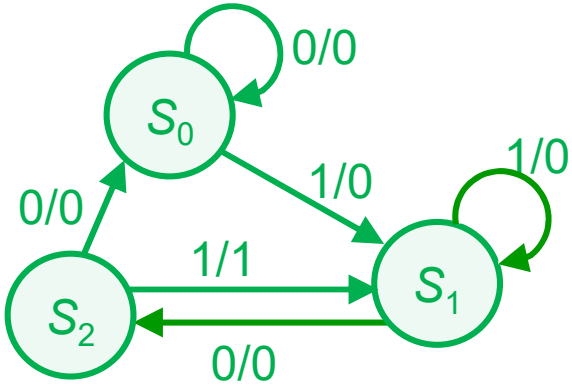
$$2^{n_F-1} < n_S \le 2^{n_F} \quad \text{or} \quad n_F = \lceil \log_2 n_S \rceil$$

Since $n_S = 3$, the number of state variables $n_F = \lceil \log_2 3 \rceil = 2$.

Let the state variables be *A* and *B*.

Present and Next State Table

Format: *X*/*Z*



Present state		I/P	Next state		O/P	Next state decoder	
A	*B*	*X*	A_{t+1}	B_{t+1}	*Z*	D_A	D_B
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	1	0	0	1
1	0	0	0	0	0	0	0
1	0	1	0	1	1	0	1
1	1	0	X	X	X	X	X
1	1	1	X	X	X	X	X

State assignment:

S_0: $AB = 00$

S_1: $AB = 01$

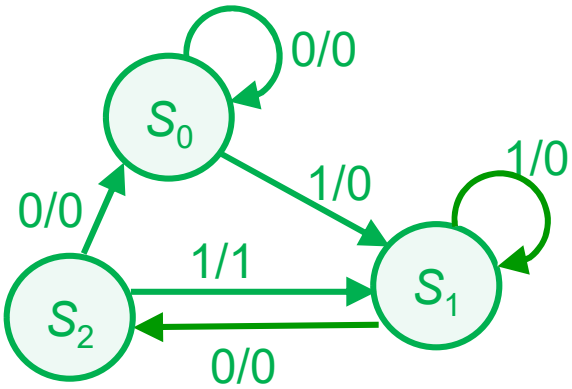
S_2: $AB = 10$

D excitation

D excitation

Derive Next State and Output Maps

Format: *X*/*Z*



Present state		I/P	Next state		O/P	Next state decoder	
A	*B*	*X*	A_{t+1}	B_{t+1}	*Z*	D_A	D_B
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	1	0	0	1
1	0	0	0	0	0	0	0
1	0	1	0	1	1	0	1
1	1	0	X	X	X	X	X
1	1	1	X	X	X	X	X

D_A

X \ AB	00	01	11	10
0	0	1	X	0
1	0	0	X	0

$$D_A = B\bar{X}$$

D_B

X \ AB	00	01	11	10
0	0	0	X	0
1	1	1	X	1

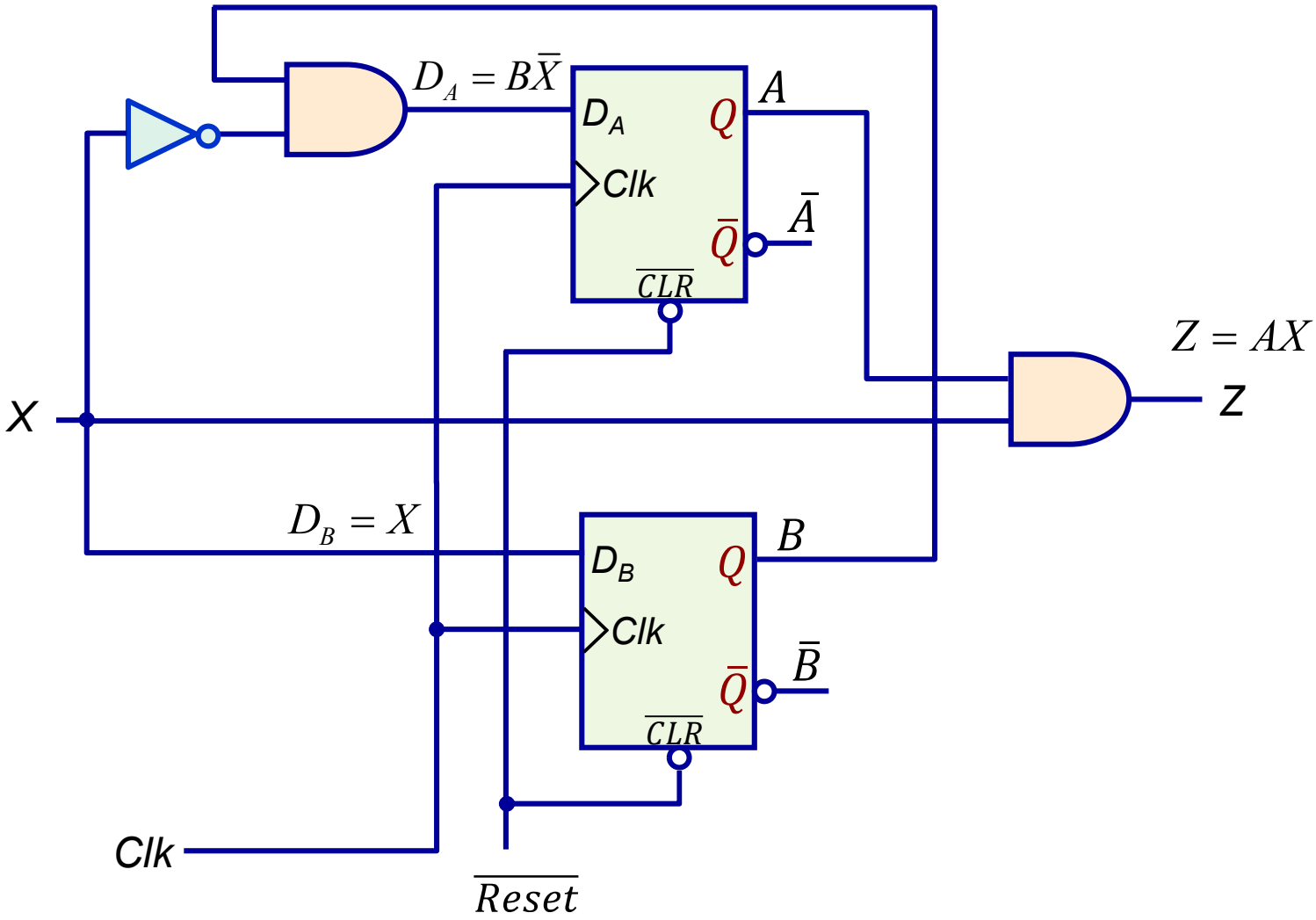
$$D_B = X$$

Z

X \ AB	00	01	11	10
0	0	0	X	0
1	0	0	X	1

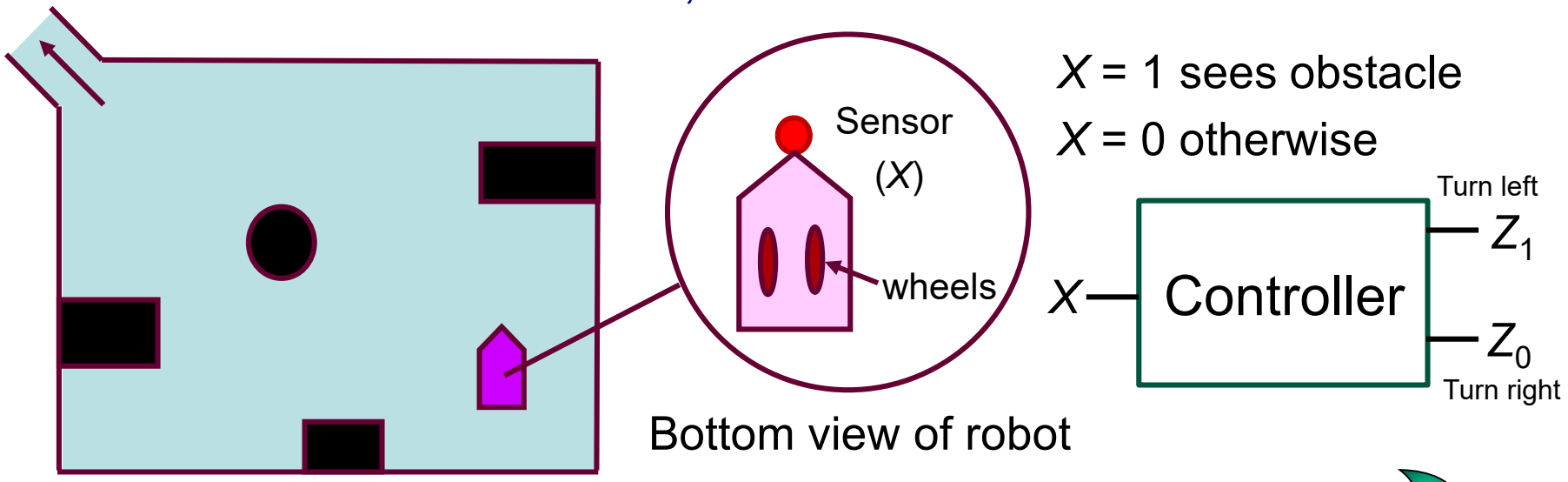
$$Z = AX$$

Draw Logic Circuit Diagram



Robot and Maze FSM Controller

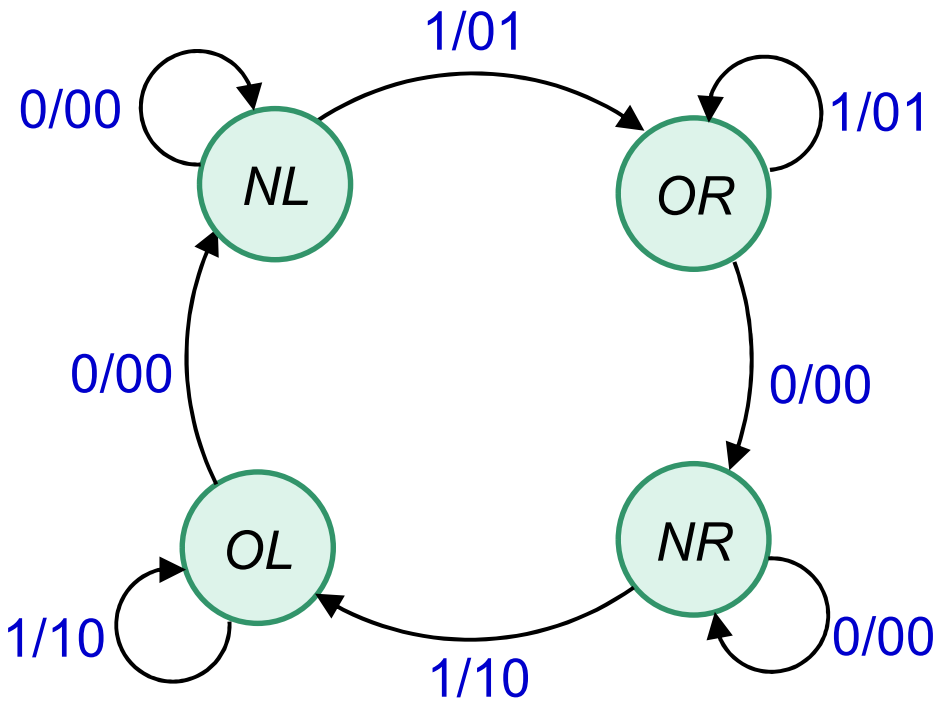
- Design a finite-state controller to maneuver the robot in the maze.
  - Robot has two control lines;
  - Z_1 =1 turns the robot to the left and Z_0 = 1 turns the robot to the right.
  - When it encounters an obstacle, the robot should turn right until no obstacle is detected.
  - The next time an obstacle is detected, the robot should turn left until the obstacle is cleared, and so on.



State Diagram and State Table

- State *NL* : no obstacle detected, last turn was left.
- State *OR* : obstacle detected, turning right.
- State *NR* : no obstacle detected, last turn was right.
- State *OL* : obstacle detected, turning left.

Format: X/Z_1Z_0

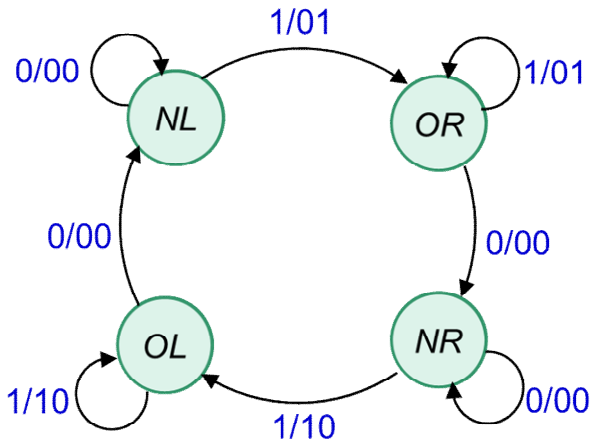


S	Input (X)	
	0	1
NL	*NL*, 00	*OR*, 01
OR	*NR*, 00	*OR*, 01
NR	*NR*, 00	*OL*, 10
OL	*NL*, 00	*OL*, 10

S^*, Z_1Z_0

Next State and Output Decoders

Format: X/Z_1Z_0



Present state		I/P	Next state		O/P		Next state code			
A	B	X	A^*	B^*	Z_1	Z_0	J_A	K_A	J_B	K_B
0	0	0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	1	0	X	1	X
0	1	0	1	1	0	0	1	X	X	0
0	1	1	0	1	0	1	0	X	X	0
1	0	0	0	0	0	0	X	1	0	X
1	0	1	1	0	1	0	X	0	0	X
1	1	0	1	1	0	0	X	0	X	0
1	1	1	1	0	1	0	X	0	X	1

JK excitation

JK excitation

State assignment:

$NL: AB = 00$

$OR: AB = 01$

$NR: AB = 11$

$OL: AB = 10$

Next State and Output Maps

PS		I/P	O/P		NS			
A	B	X	Z_1	Z_0	J_A	K_A	J_B	K_B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	0	0	1	X	X	0
0	1	1	0	1	0	X	X	0
1	0	0	0	0	X	1	0	X
1	0	1	1	0	X	0	0	X
1	1	0	0	0	X	0	X	0
1	1	1	1	0	X	0	X	1

$J_A = B\bar{X}$

X \ AB	00	01	11	10
0	0	1	X	X
1	0	0	X	X

$K_A = \bar{B}\bar{X}$

X \ AB	00	01	11	10
0	X	X	0	1
1	X	X	0	0

$J_B = \bar{A}X$

X \ AB	00	01	11	10
0	0	X	X	0
1	1	X	X	0

$K_B = AX$

X \ AB	00	01	11	10
0	X	0	0	X
1	X	0	1	X

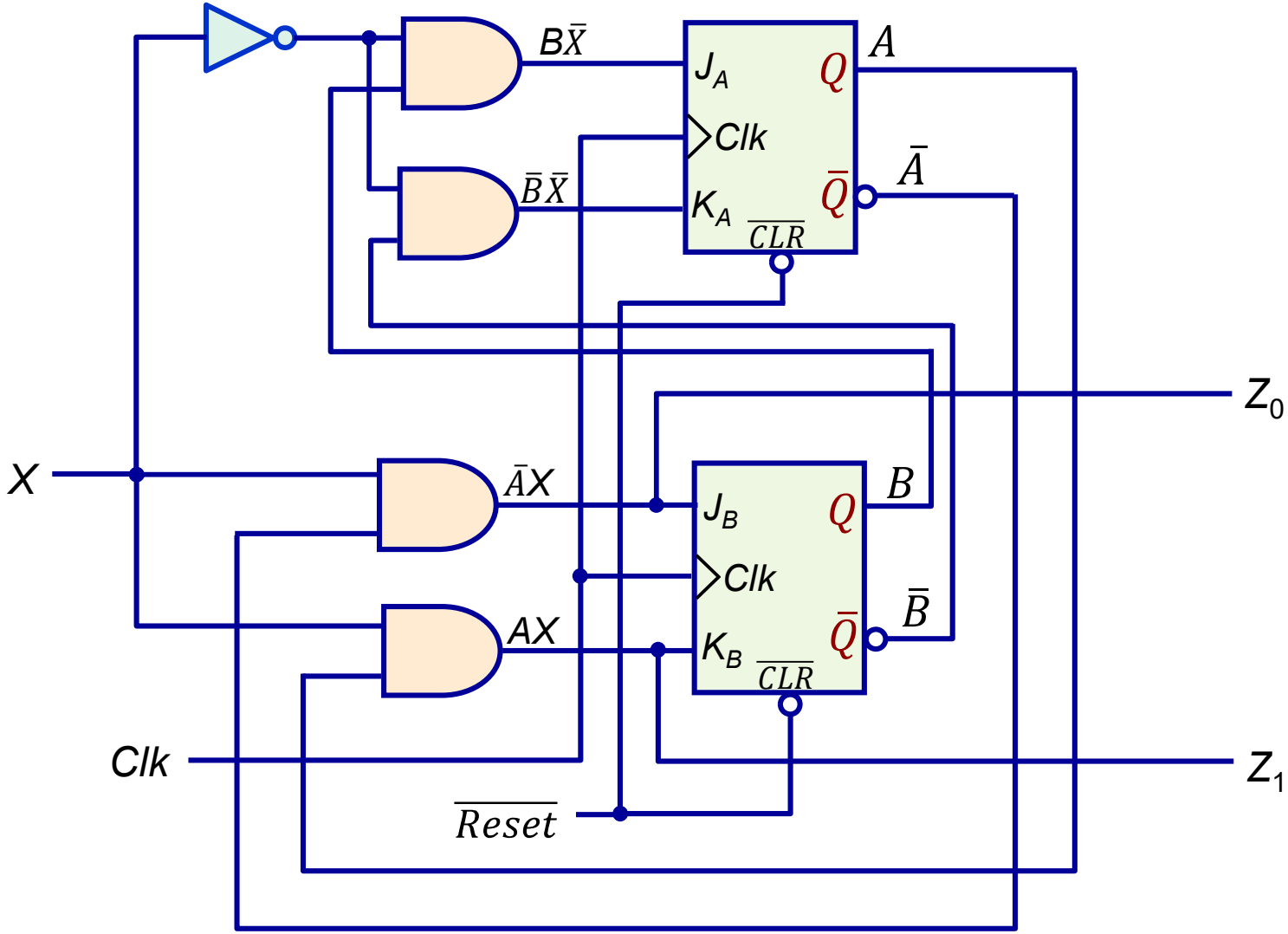
$Z_1 = AX = K_B$

X \ AB	00	01	11	10
0	0	0	0	0
1	0	0	1	1

$Z_0 = \bar{A}X = J_B$

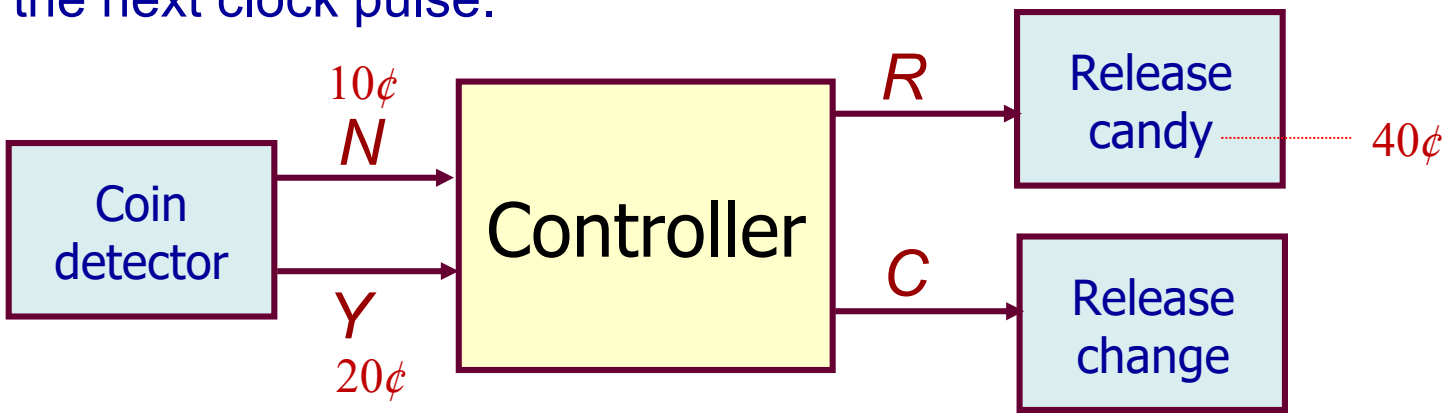
X \ AB	00	01	11	10
0	0	0	0	0
1	1	1	0	0

Draw Logic Circuit Diagram



Simple Vending Machine Controller

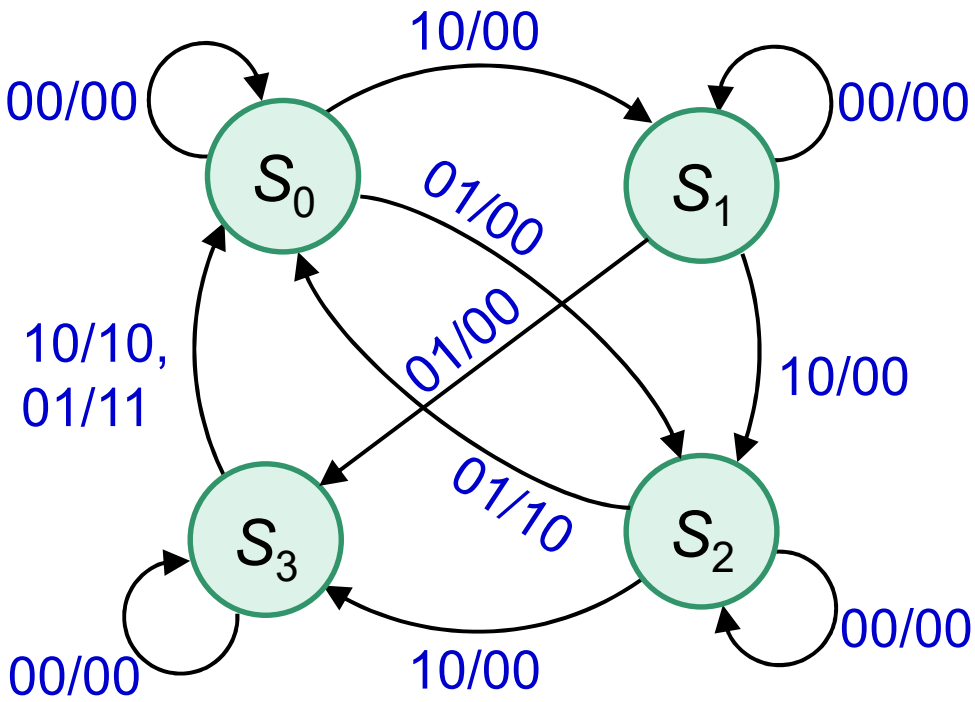
- Design a controller for a simple coin-operated candy machine.
  - A candy cost 40¢, and the machine could only accept 10¢ and 20¢ coins.
  - Change should be returned if more than 40¢ is deposited.
  - No more than 50¢ can be deposited on a single purchase; therefore, the maximum change is 10¢;
  - Assume that it is physically impossible to insert two coins at the same time.
  - Also assume the coin detector automatically reset its outputs to 0 in the next clock pulse.



Construct State Diagram

- *N* = 1 if a 10¢ coin is deposited, *Y* = 1 if a 20¢ coin is deposited.
- *R* = 1 releases the candy, *C* = 1 releases the change.
- *N* = *Y* = 1 in the same clock cycle is impossible ⇒ NY = 11 not allowed.

Format: *NY*/*RC*

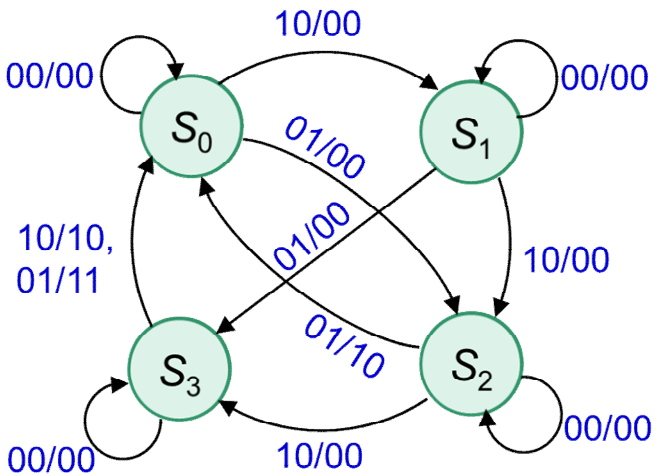


State assignment:

S_0 : *AB* = 00 (0¢ deposited)
S_1 : *AB* = 01 (10¢ deposited)
S_2 : *AB* = 10 (20¢ deposited)
S_3 : *AB* = 11 (30¢ deposited)

Next State and Output Decoders

Format: *NY/RC*



State assignment:

S_0 : AB = 00
S_1 : AB = 01
S_2 : AB = 10
S_3 : AB = 11

Present state		Inputs		Next state		Outputs		F/F inputs	
A	B	N	Y	A^*	B^*	R	C	T_A	T_B
0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0	1	0
0	0	1	0	0	1	0	0	0	1
0	0	1	1	X	X	X	X	X	X
0	1	0	0	0	1	0	0	0	0
0	1	0	1	1	1	0	0	1	0
0	1	1	0	1	0	0	0	1	1
0	1	1	1	X	X	X	X	X	X
1	0	0	0	1	0	0	0	0	0
1	0	0	1	0	0	1	0	1	0
1	0	1	0	1	1	0	0	0	1
1	0	1	1	X	X	X	X	X	X
1	1	0	0	1	1	0	0	0	0
1	1	0	1	0	0	1	1	1	1
1	1	1	0	0	0	1	0	1	1
1	1	1	1	X	X	X	X	X	X

Next State and Output Maps

PS		I/P		O/P		F/F I/P	
A	*B*	*N*	*Y*	*R*	*C*	T_A	T_B
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	1
0	0	1	1	X	X	X	X
0	1	0	0	0	0	0	0
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	X	X	X	X
1	0	0	0	0	0	0	0
1	0	0	1	1	0	1	0
1	0	1	0	0	0	0	1
1	0	1	1	X	X	X	X
1	1	0	0	0	0	0	0
1	1	0	1	1	1	1	1
1	1	1	0	1	0	1	1
1	1	1	1	X	X	X	X

NY \ AB	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	X	X	X	X
10	0	1	1	0

$$T_A = Y + BN$$

NY \ AB	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	X	X	X	X
10	1	1	1	1

$$T_B = N + ABY$$

NY \ AB	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	X	X	X	X
10	0	0	1	0

$$R = AY + ABN$$

NY \ AB	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	X	X	X	X
10	0	0	0	0

$$C = ABY$$

Draw Logic Circuit Diagram

